

Advanced Perception for Autonomous Platforms in the Littorals

Contract # N00014-23-C-1052

Monthly Scientific and Technical Report 22
Covered Period: May 1, 2025 – May 31, 2025

Prepared by

University of South Carolina
Yi Wang, Yan Tong, Sanjib Sur, Ramtin Zand, David Matolak, Douglas Cahl, and Chadwick Severt

Integer Technologies, LLC
Tristan Kyzer, Phil Stone, Josh Knight, and Duke Hartman

Prepared for

Office of Naval Research (ONR)
Program Managers: Christine Buzzell and Dr. Robert Brizzolara

Technical Contact:
Name: Yi Wang
Phone: 803-777-2252
Cell: 256-652-3253
Email: yiwang@cec.sc.edu

Financial Data/Contract Contact:
Name: Brandi Boniface
Phone: 803-777-8749
Email: bonifacb@mailbox.sc.edu

Administrative Contact:
Name: Danielle McElwain
Phone: 803-777-2094
Email: dmcclwai@cec.sc.edu



Distribution D - Distribution authorized to the Department of Defense and U.S. DoD contractors only; Critical Technology; 01 08 2021. Other requests for this document shall be referred to the ONR Code 33 (Christine Buzzell, Program Officer, christine.m.buzzell.civ@us.navy.mil).

Table of Contents

1. Introduction and Project Summary.....	3
2. Progress.....	3
2.1 Overall Progress Summary.....	3
2.2 Individual Task Progress.....	4
2.2.1 Geometric Range Estimation.....	4
2.2.2 Longwave Infrared (LWIR) Range Estimation.....	6
2.2.3 FLIR Slue to Cue Development and Initial Results.....	8
2.2.4 Deraining Model Improvements.....	10
2.2.5 IMSEL Maritime Dataset Development.....	15
2.2.6 SIMRAD ROS2 Node Development.....	19
3. Problems.....	22
3.1 Encountered Problems.....	22
3.2 Anticipated Problems.....	22
3.3 Resource Utilization.....	22
4. Next Reporting Period Actions.....	22
5. References.....	24

1. INTRODUCTION AND PROJECT SUMMARY

This is a Monthly Scientific and Technical Report for an ONR project entitled Advanced Perception for Autonomous Platforms in the Littorals ". The objectives of the effort in this period were (1) calibration, rectification, and field deployment of the M364C FLIR camera system to enhance IR-RGB alignment and expand the IMSEL Maritime Dataset to 8,589 annotated image-label pairs; (2) develop a fusion-based approach to reliably detect small, low-RCS objects at sea by combining radar and camera data, and use radar cues to streamline image labeling.; (3) focused on accuracy testing and algorithmic improvements for the orthorectification process from the camera system used for surface current measurements; (4) fusing radar camera and INS data on the Jetson Orin; (5) rain segmentation and data cleaning for the deraining task.; (6) optimal camera calibration for different geometric range distance estimates; (7) increase robustness of long wave IR range estimates; (8) assess deep learning as a method for range estimates; (9) assess uncertainty in the geometric range estimates; and (10) create a virtual ocean environment with shorelines approaches for model training and real time shoreline characterization.

2. PROGRESS

2.1 Overall Progress Summary

Efforts during this work period (May 1st – May 31st, 2025) have focused on several aspects:

- (1) We outline the calibration, rectification, and deployment processes of the M364C FLIR camera system for enhancing the IMSEL Maritime Dataset. Multiple calibration approaches including MATLAB Stereo Calibrator, uncalibrated rectification, and both manual and automated homography were evaluated to align IR and RGB image streams. A thermal calibration board was developed to improve checkerboard visibility in the IR spectrum. Field data was captured near Charleston, SC contributing to a current total of 8,589 annotated image-label pairs, bringing the dataset within 1,411 images of its 10,000-image target. These efforts are important for improving the accuracy of computer vision models in challenging maritime environments, particularly for detecting small objects in low-light and multi-modal imagery.
- (2) Conventional marine radars alone struggle to detect small, low radar-cross-section (RCS) objects (like kayaks, swimmers, buoys, pylons) in higher sea states and heavy sea clutter, and optical camera detectors are limited by scarce labeled data in marine settings. By fusing X-band radar and cameras, the system will improve the detection of small maritime objects and automatically assist in labeling camera images using radar returns as cues.
- (3) We focused on accuracy testing and algorithmic improvements in the orthorectification process. Testing was conducted using images captured in front of the Swearingen Engineering Building, where standardized concrete tiles provided a known reference for evaluating rectification accuracy. Measurements from rectified images revealed minor edge length variations, with all observed errors remaining below 10%. Additionally, a significant pipeline enhancement was implemented by replacing fixed-point image cropping with a world coordinate distance cutoff method. This approach prevents errors related to projecting distant pixels and eliminates the need for manual cutoff tuning. The system now identifies edge pixels that exceed a specified world distance and uses the resulting points to define the bounds of the rectified output. This improvement increases both the reliability and scalability of the orthorectification process.
- (4) This report details the implementation of raw radar data saving on the Jetson AGX Orin, completing the fusion of radar, camera, and INS sensors into a unified data collection platform. Timestamped radar sweeps are now stored alongside synchronized camera images and INS metadata in a structured format suitable for downstream processing. Real-time validation confirmed accurate operation across all streams, with no data loss or misalignment. The resulting dataset enables precise time-correlated analysis for sensor fusion applications.

- (5) We trained a rain segmentation model to localize raindrops in rain images and improved our data cleaning procedure for dataset creation to further improve our rain segmentation model. The rain segmentation model leverages a state-of-the-art backbone for rain segmentation. The data cleaning step seeks to improve the quality of our ground truth labels to prevent incomplete and incorrect labels when training segmentation models.
- (6) We successfully experimented with different camera calibration methods during this reporting period and determined that the best method is when the calibration board is farther away from the camera (about 10m). We have also created a method of calibrating the camera and IMU together and generated the transformation matrix and time shift between the two devices.
- (7) In this month's reporting period, the team re-parameterized independent variables to train and test XGBoost in order to bypass gaps in important metadata that are missing from the open-source dataset. The robustness of the algorithm was improved, increasing the R^2 value from 0.65 to 0.90 when utilizing training data from 7 different datasets across 14 different unknown sensor-gain values. Additionally, the team demoed a state-of-the-art LWIR sensor from Axiom Optics, collecting more than 250 maritime images paired with a ground truth distance to objects in-frame.
- (8) This month, Integer worked on assessing deep learning methods as a potential alternative for outdoor, open scene range estimation. Integer also explored the impact of camera orientation in model performance.
- (9) In this reporting period, the Integer Team looked at uncertainty associated with estimating range from images using the Ray Tracing approach to better understand the fidelity of range estimates made and the limitations of the approach. This analysis will help drive system modifications that could be effective in increasing the fidelity of the measurements.
- (10) This month, Integer created a virtual ocean environment on Blender with a boat mounted camera to simulate scenarios of approaching shoreline. This technique generates segmentation training data in the form of a binary mask for each object, enabling training of segmentation models like SegFormer, for real time shoreline characterization.
- (11)

2.2 Individual Task Progress

2.2.1 Enhancing Maritime Object Detection: Calibration, Rectification, and Dataset Expansion with the M364C FLIR Camera

Part 1: Low-light camera GUI

The Camera GUI for both the M364C FLIR and Nightwave cameras has been significantly updated. New features include a program uptime display and real-time system metrics from the Jetson Orin, such as CPU usage, RAM usage, and system temperature. The "time remaining until storage is full" indicator has been simplified and is now presented in hours. Additionally, new view options have been introduced, allowing users to select individual camera feeds, enable full-screen viewing, and overlay the FLIR camera's infrared (IR) stream on top of the RGB stream for enhanced visualization as shown in Figure 1.

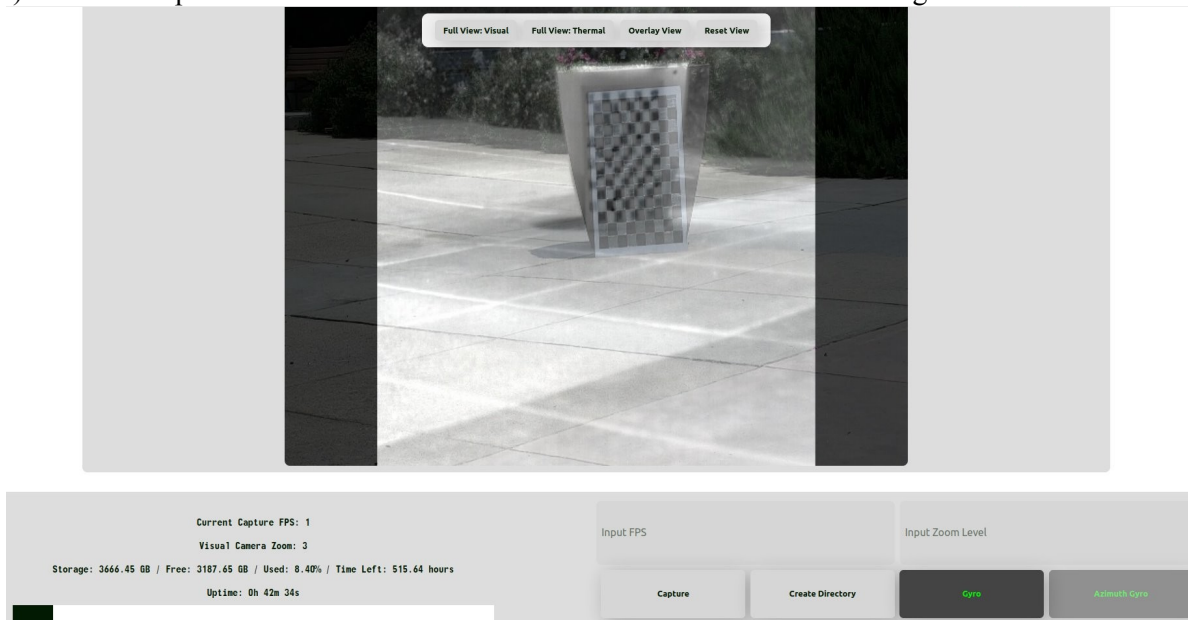


Figure 1: Low-Light Camera GUI

Part 2: Thermal Calibration Board

To enable thermal detection of the calibration pattern, aluminum squares were affixed over the existing white squares on the lab's calibration board. Each aluminum square measures 60 mm in size. Due to aluminum's low emissivity its limited ability to emit thermal radiation the FLIR camera is now able to clearly detect the checkerboard pattern in the infrared (IR) stream as seen in Figure 2. Previously, the pattern was not visible through thermal imaging.

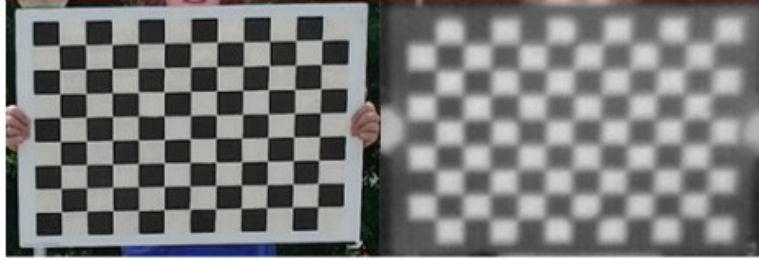


Figure 2: Camera Calibration Board with AL

Using the newly constructed calibration board, multiple data captures were conducted to obtain high-quality calibration data. Such a test can be seen conducted in Figure 3.



Figure 3: Calibration Board Capture

The new overlay feature in the camera GUI simplified the process of approximating which optical zoom level of the RGB feed best aligns with the IR stream, as shown in Figure 4. The corresponding pixel-wise differences for each zoom level selection are also illustrated.

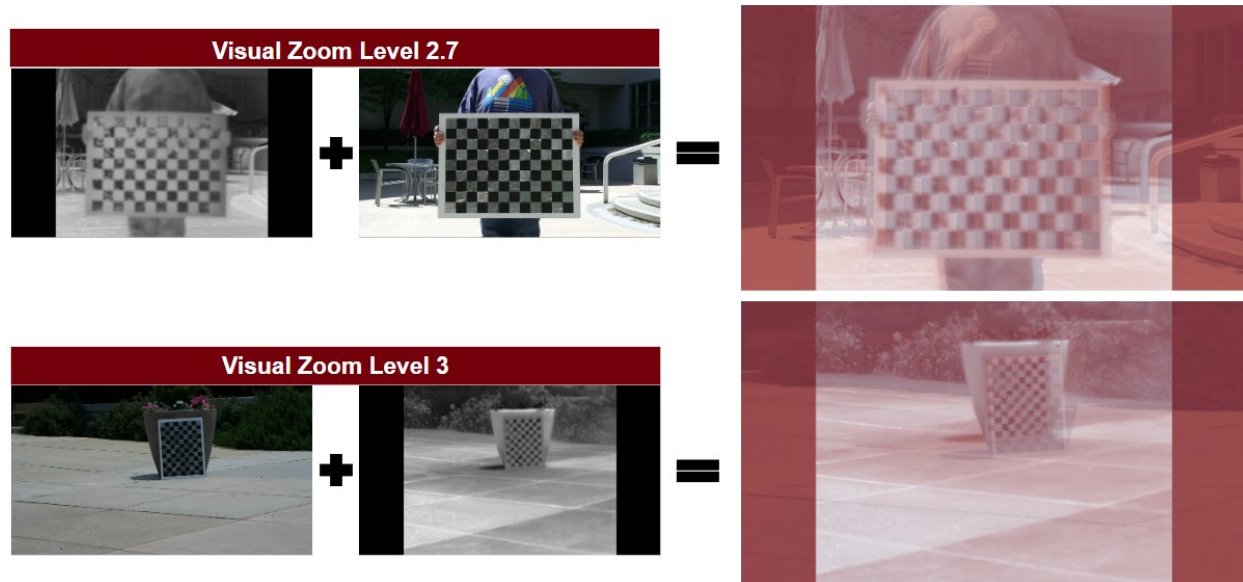


Figure 4: Varying Zooms with Overlay and Pixel-wise difference

Part 3: M364C Camera Calibration – MATLAB Stereo Camera Calibrator

Using the captured calibration board data, paired IR and RGB frames were processed through the MATLAB Stereo Camera Calibrator. The calibration successfully reduced the mean reprojection error to 0.80 pixels. However, the resulting rectification was not satisfactory, indicating potential issues inherent to RGB vs IR. The results can be seen in

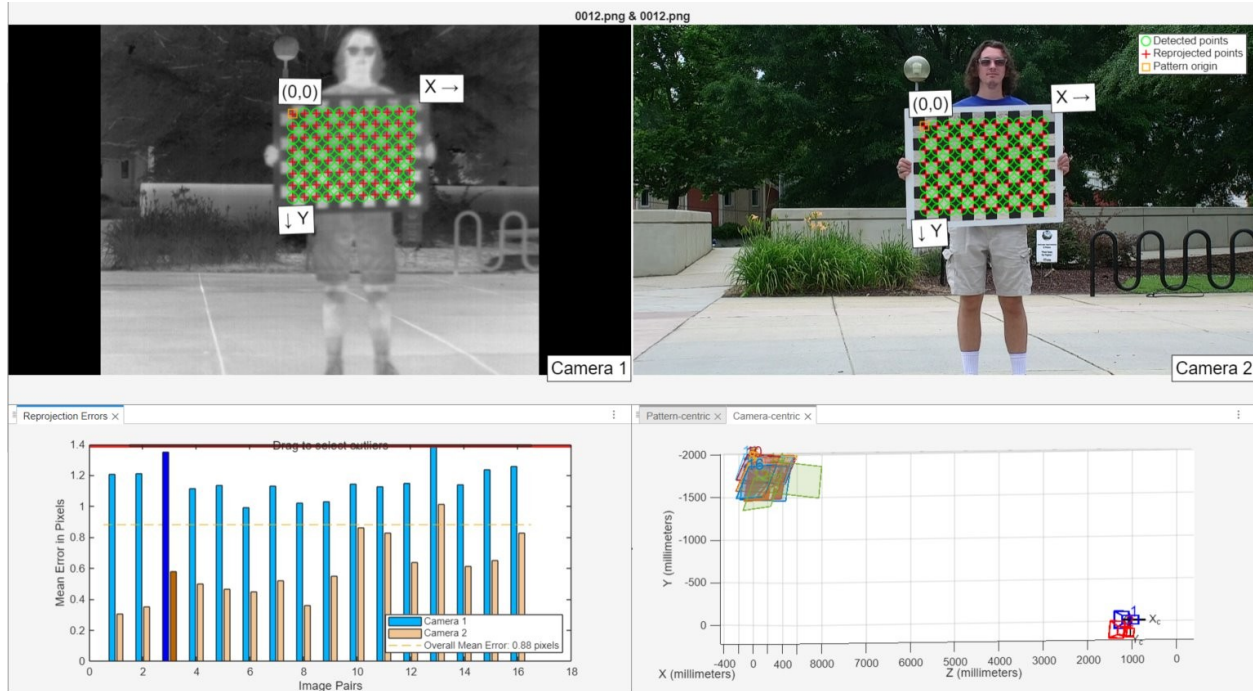


Figure 5: MATLAB Stereo Camera Calibrator



Figure 6: MATLAB Stereo Camera Calibrator Results

Part 4: M364C Camera Calibration – Uncalibrated Stereo Image Rectification

As an alternative to traditional calibration methods, uncalibrated stereo image rectification was explored. This approach allows for the rectification of image pairs without requiring prior intrinsic or extrinsic calibration. The process began by computing the pixel-wise difference between the paired IR and RGB images to evaluate alignment and identify areas of disparity as seen in Figure 7.



Figure 7: Pixel-Wise Difference

Next, the thirty strongest SURF (Speeded-Up Robust Features) points were detected and overlaid on the paired IR and RGB images as seen in Figure 8. SURF identifies distinctive keypoints within an image, such as corners and blobs, which are robust to changes in scale and rotation. These keypoints served as the foundation for estimating geometric transformations between the two image streams.

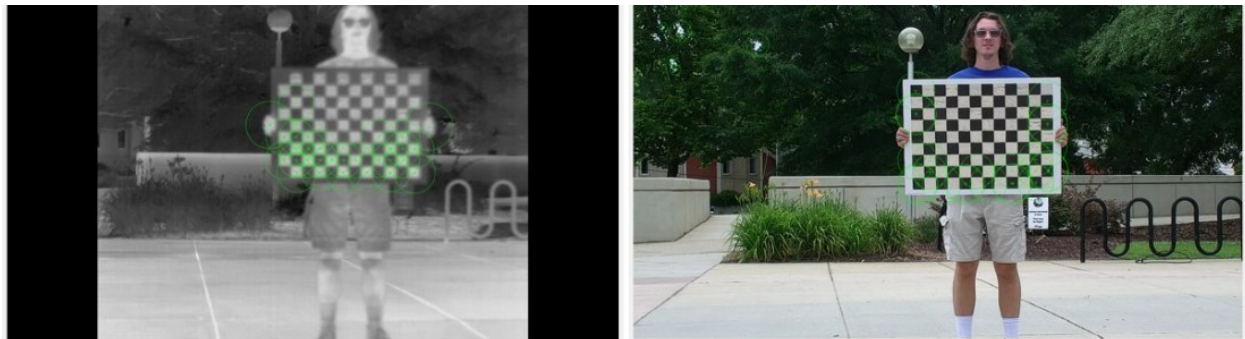


Figure 8: SURF Points

The next step in the process involved detecting putative point correspondences initial feature matches between the IR and RGB images based on visual similarity. However, this step failed, as no putative correspondences were successfully identified. As a result, the uncalibrated stereo image rectification method was unable to proceed, and the image pair could not be rectified using this approach.

Part 5: M364C Camera Calibration – Python Manual Homography

Homography calibration involves computing a transformation matrix that maps points from one image to another, enabling one image to be warped to align with the perspective of another. This technique is particularly effective for planar scenes, making the checkerboard an ideal reference pattern.

The transformation follows the equation:

$$\begin{bmatrix} x' \\ y' \\ 1 \end{bmatrix} = H \cdot \begin{bmatrix} x \\ y \\ 1 \end{bmatrix}$$

where (x, y) are coordinates in the original image, (x', y') are the corresponding coordinates in the target image, and H is the homography matrix. To implement this, a custom Python script was used that enabled manual selection and labeling of matching points between the IR and RGB images. The point selection

can be seen in Figure 9. These correspondences were then used to compute the homography matrix and rectify the RGB frame to align with the IR frame. This method provided some fairly good results with the checkered board as seen in Figure 10.



Figure 9: Point Selection

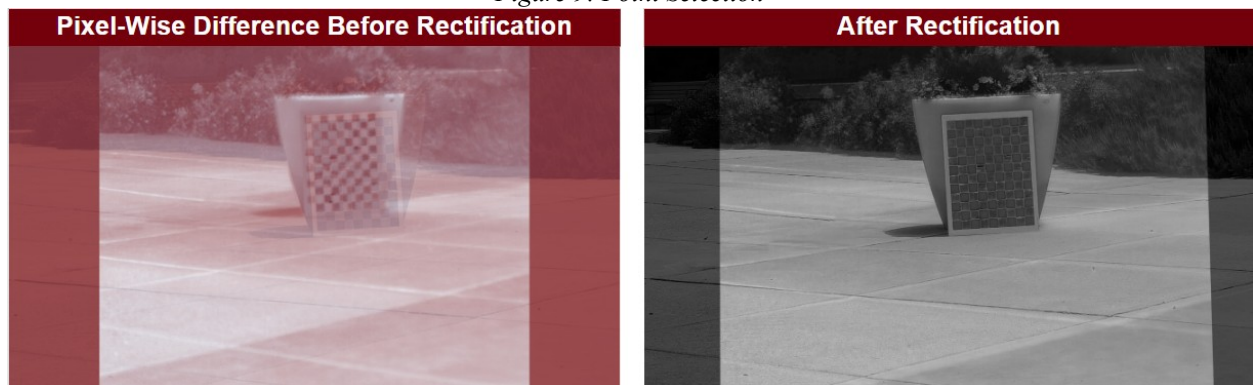


Figure 10: Manual Homography Results

Part 6: M364C Camera Calibration – MATLAB Automated Homography



Figure 11: MATLAB Automated Homography Pipeline

Figure 11 above illustrates the processing pipeline used for the automated homography in MATLAB.

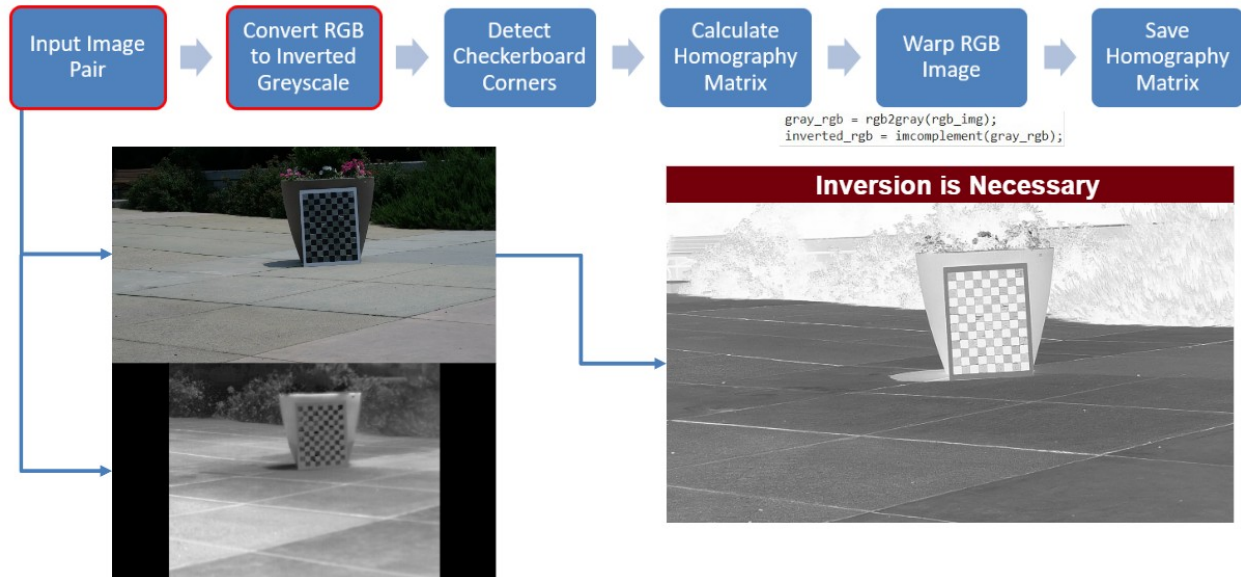


Figure 12: Pipeline Part 1

The pipeline begins in Figure 12 with importing a pair of input images. Then the RGB image is converted to an inverted greyscale image as without this feature the corner detector will think that the RGB checkered board is horizontally flipped.

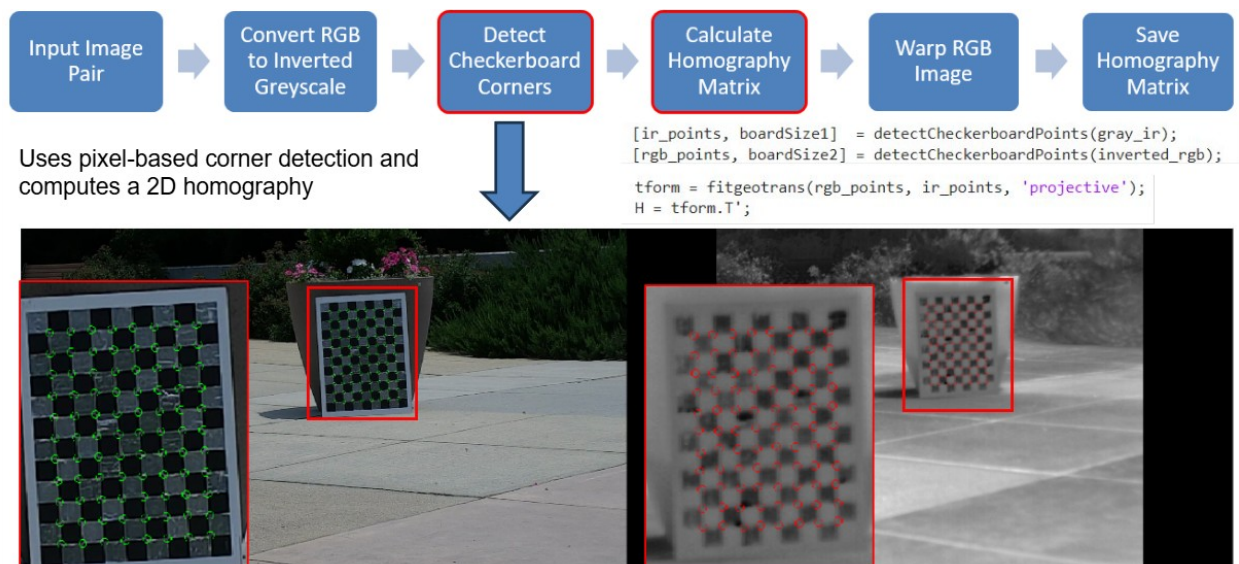


Figure 13: Pipeline Part 2

Next in the pipeline, Figure 13, MATLAB's built in corner detection functions are used to identify the checkerboard corners in both images. Once corresponding points are established, the homography matrix is calculated. This matrix defines the projection transformation require to warp the RGB image so that it aligns with the IR frame.

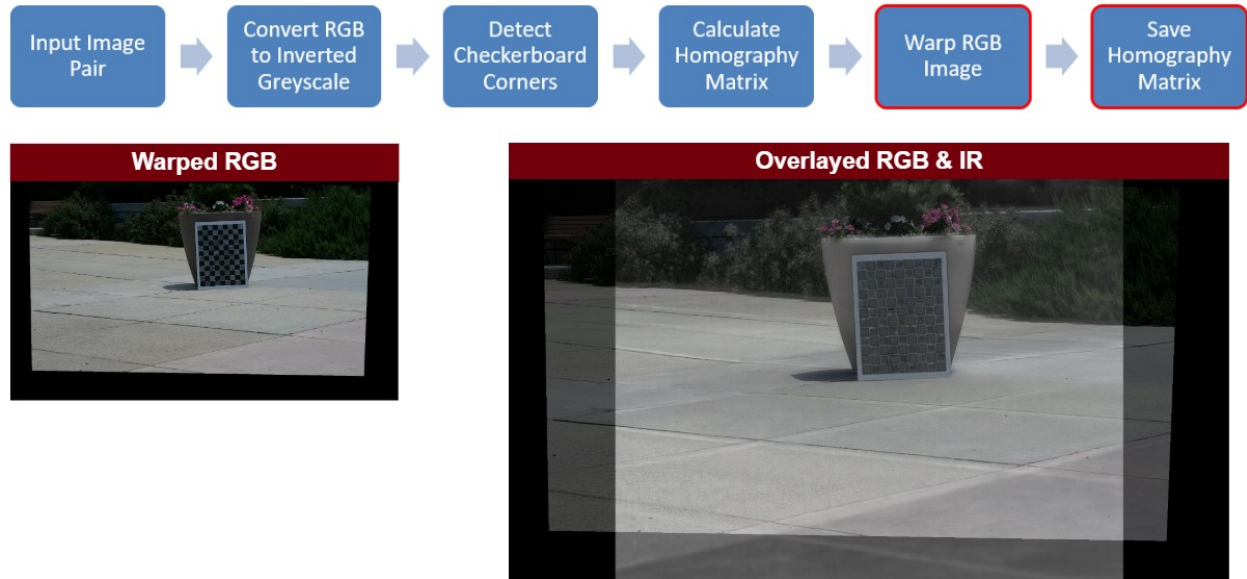


Figure 14: Pipeline Part 3

Finally in Figure 14 the RGB image is geometrically warped based on this generated homography, and the resulting transformation matrix is saved for future use. This automated method improves consistency and speed compared to the manual homography.

Part 7: Data capture – Location Downtown Charleston

On June 6th, a data capture event was conducted in downtown Charleston using the M364C FLIR camera system. The goal of this field session was to collect diverse real-world imagery under maritime-relevant conditions. The captured data was subsequently processed and incorporated into the IMSEL Maritime Dataset.



Figure 15: Charleston 6/6/25

Part 8: IMSEL Maritime Dataset

As of the latest update, the IMSEL Maritime Dataset contains a total of 8,589 matched image-label pairs, bringing it significantly closer to the target benchmark of 10,000 annotated images. With only 1,411 pairs remaining, ongoing data collection and annotation efforts will see this goal reached very soon. Updated metrics for class frequency and the annotation instances across versions can be found in Figure 16.

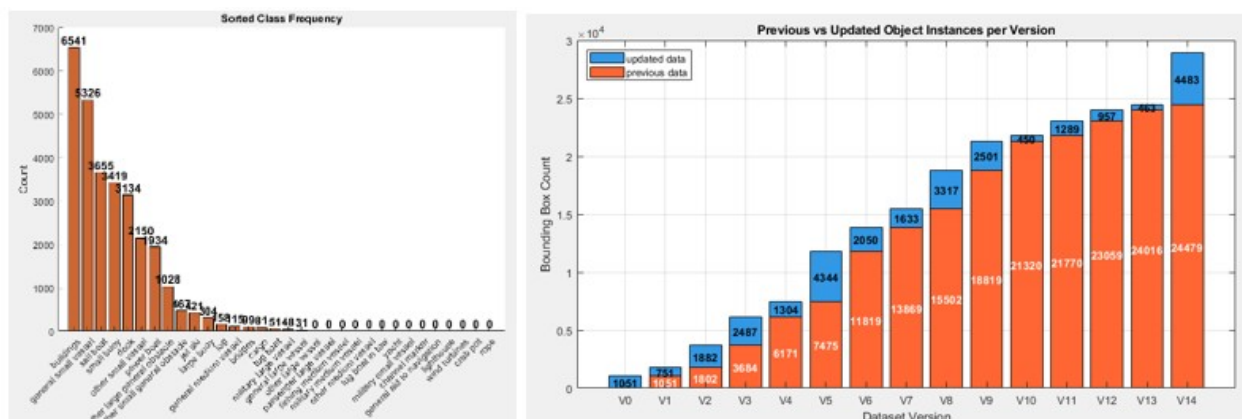




Figure 17: Small Object Detection: A Succinct Review

Existing public maritime image datasets are few, and those that do exist have limited scale and diversity. Deep learning models demand plentiful labeled examples to achieve high accuracy, yet acquiring those examples on the open ocean is impractical. As a result, current vision models trained on sparse maritime data struggle to detect small, distant objects reliably. In comparison to common object detection on land, labeling maritime scenes (mostly water with tiny far-off targets) is far more costly and difficult. This shortage of annotated data is a major barrier to deploying robust camera-based detectors on marine platforms.

Secondary sensor suites, such as marine radar systems, often have difficulty reliably detecting very small, low-RCS targets (e.g. a partially submerged person, small buoy, or debris) against the dynamic ocean background. Such objects return very weak echoes that are easily masked by sea clutter, the radar returns from waves. In practice, the unfavorable signal-to-clutter ratio means that small craft or persons in the water may go undetected by conventional X-band navigation radars in rough seas. Even specialized surveillance radars (for example, airborne periscope-detection radars) suffer high false-alarm rates in choppy conditions, as breaking wave tops produce spurious bright clutter spikes that can mimic or eclipse genuine targets. In short, using radar alone, small maritime objects are often lost in clutter or generate many false detections. An example of Low-RCS objects in differing clutter environments is provided in Figure 18.

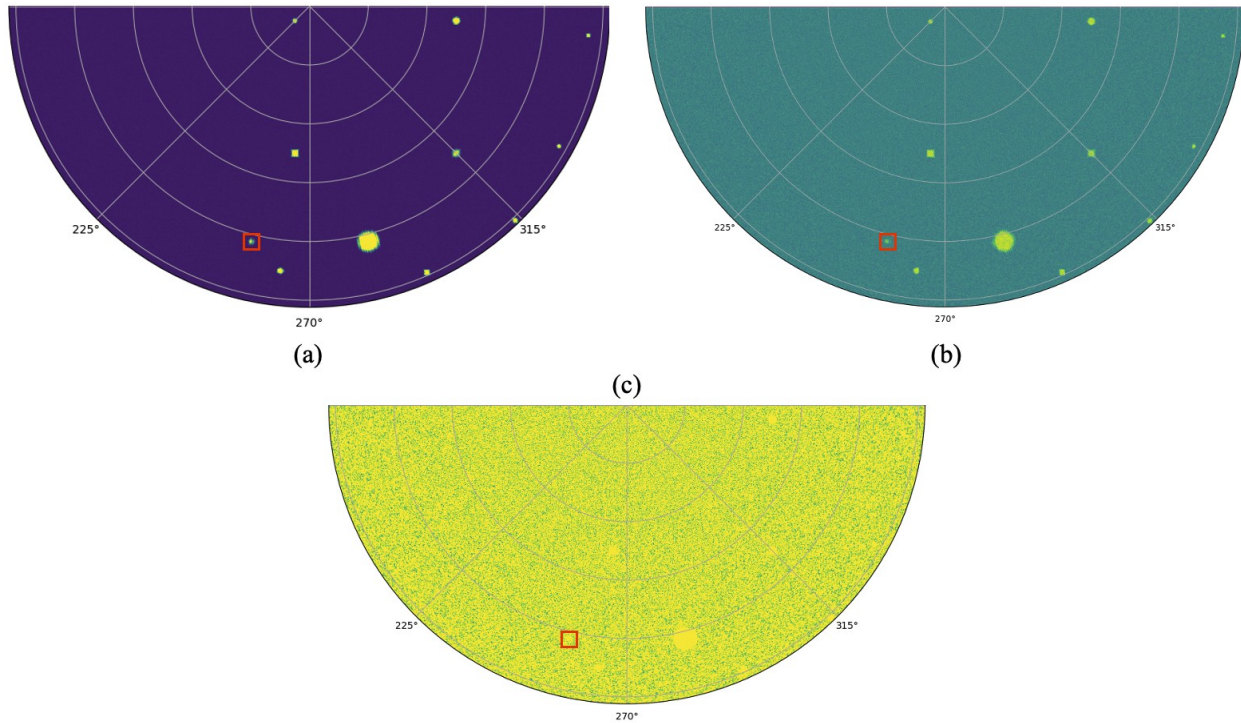


Figure 18: Low-RCS Objects in Low, Med, & High Rain Clutter

This represents a critical gap for maritime safety and security, as low-profile hazards or intruders may elude single-sensor detection. However, Radar-camera sensor fusion offers a promising solution to overcome these challenges. Marine radar operates day or night and in fog, rain, or glare where optical cameras often fail, and can detect objects at long range beyond the visible horizon. A preliminary sketch of this proposal is seen in Figure 19.

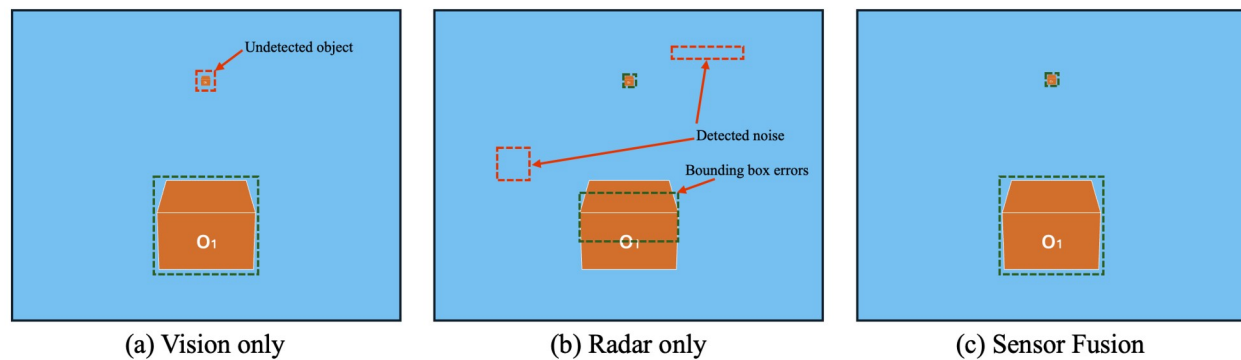


Figure 19: Comparison of Camera and Radar detection systems

By leveraging radar’s all-weather detections to cue the camera, we can effectively “seed” the image labeling process. For instance, if the radar returns indicate a target at a certain range and bearing, that can flag a region-of-interest in the corresponding camera frame for a human annotator (or an assisting algorithm) to verify and label. Researchers have begun exploring such cross-sensor cueing; for example, a Radar Region Proposal Network (RRPN) has been proposed to project radar detections into the camera view as candidate object regions. In our context, the radar will act as an automatic proposal generator for the camera, focusing attention on likely object locations, especially valuable when objects are small or visibility is poor. The overarching goal (and motivation for this project) is to bridge the radar and optical modalities to overcome their individual limitations. A fusion of radar and camera information promises more robust detection of small maritime objects and a means to automate data labeling. Radar can

compensate for the camera's shortcomings in low-visibility conditions, while the camera provides high-resolution classification that radar lacks. By jointly using radar and optical sensors, we aim to improve the detection performance for small objects at sea and develop an automated labeling aid; addressing both the detection gap of radar and the data scarcity problem of deep learning models. One possible annotator-assisted pipeline is shown in Figure 20; combining 2 separate algorithms to create useful annotation data.

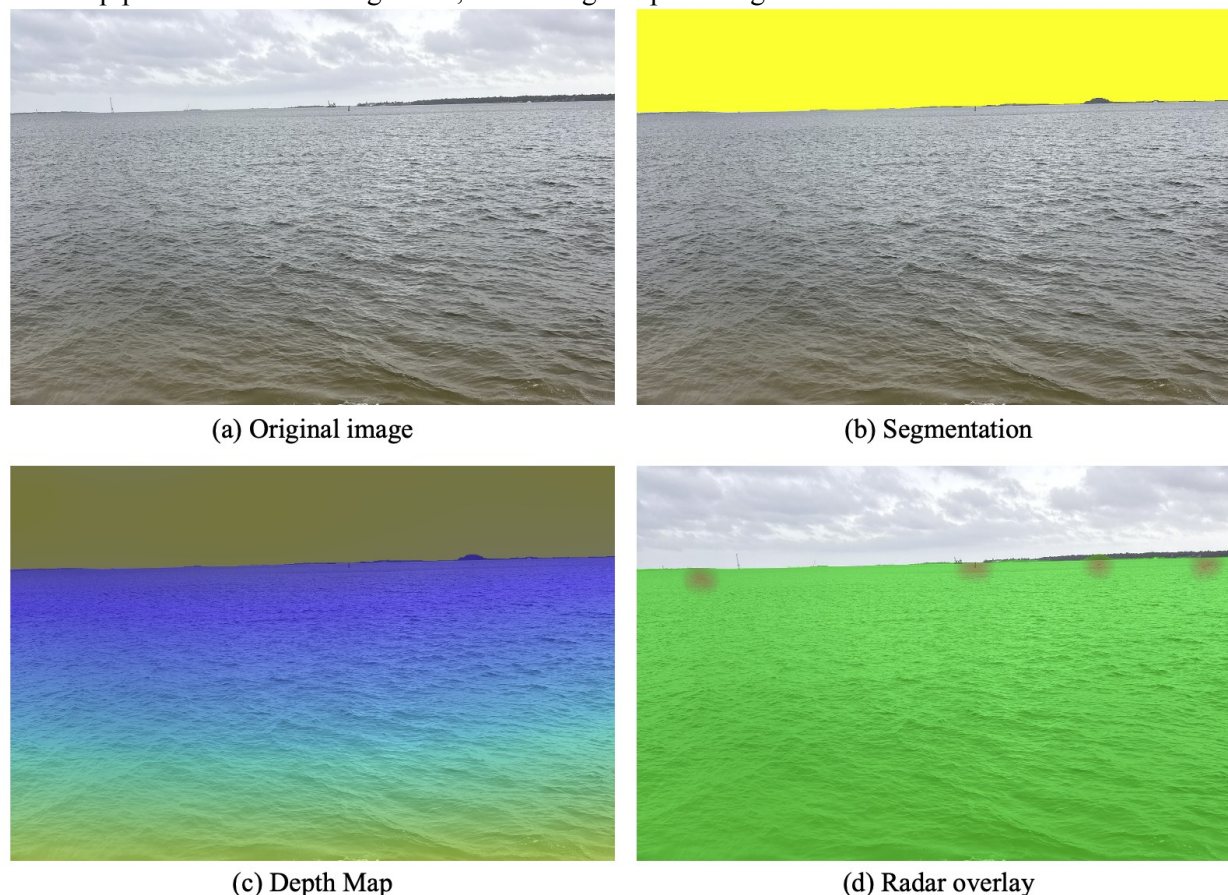


Figure 20: Radar-Overlay Object Detection Pipeline

Part 2 Technical Background

Part 2.1 Marine Radar Fundamentals

Modern marine radars operate using either a pulsed or Frequency-Modulated Continuous-Wave (FMCW) scheme. Traditional pulsed radar (often magnetron-based) transmits high-power microwave pulses and measures the echo return time to determine target range. These can emit kilowatt-level peak power pulses, offering long-range detection, but they require a warm-up period and have a short blind zone (minimum range) during each pulse when the radar cannot receive. In contrast, FMCW radar continuously sweeps its transmit frequency and listens for the frequency-shifted returns, achieving similar range performance with much lower peak power (on the order of tens of watts). An example of the difference between FMCW and Pulse radar is shown in Figure 21. FMCW radars have no inherent blind range (since they transmit and receive simultaneously) and provide improved short-range detection and Doppler sensitivity due to their coherent processing. While magnetron pulse radars remain popular for their simplicity and long-range capability, solid-state FMCW radars are increasingly adopted for their instant-on operation, lower power consumption, and ability to detect very close-in objects that pulsed radars might miss.

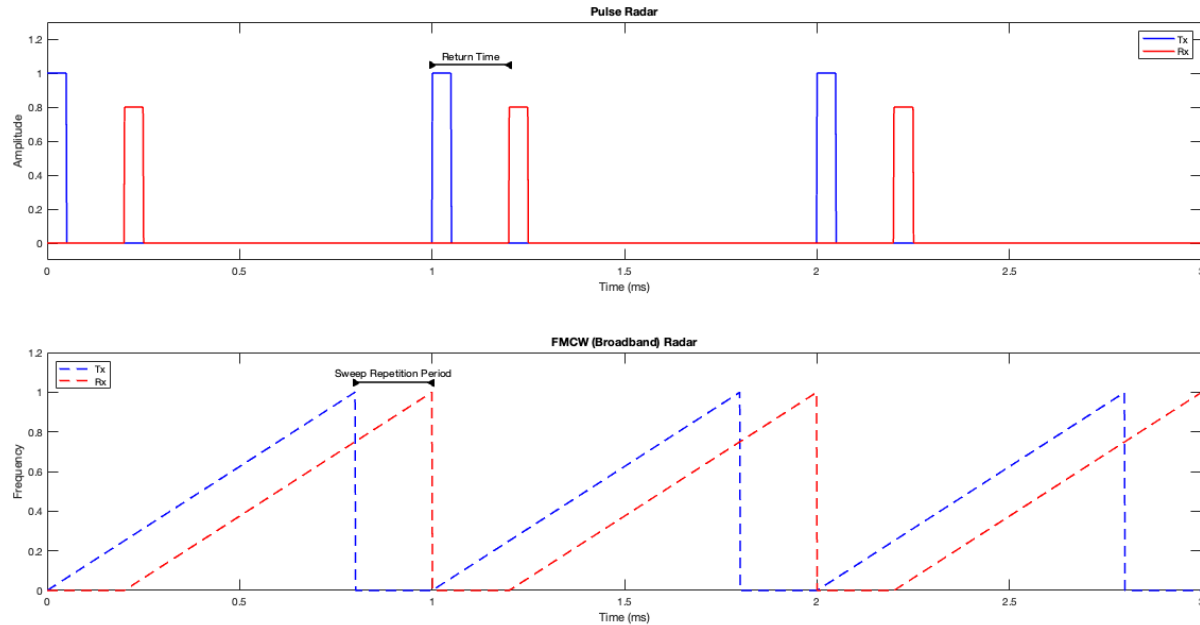


Figure 21: Comparison of Pulse and FMCW Radar Waveforms

For this project, an X-band marine radar (9.4 GHz) is utilized due to low-cost, power efficiency, and ease-of-use. Regardless of the chosen architecture, understanding radar operating mode (pulsed vs. FMCW) will inform how collected data is to be processed and integrated with other sensors or cameras. Although some radar systems (automotive primarily) are stationary; the majority of marine radar provides a 360° panoramic image of echoes (often referred to as PPI or Plan Position Indicator) in which targets are detected against sea clutter.

Part 2.2 Gain Control and CFAR

A key challenge in radar processing is distinguishing small targets from sea clutter and noise, which requires careful setting of detection thresholds. Radar units include gain control circuits to adjust the amplification of incoming echoes, effectively controlling the display intensity of returns. Proper gain setting is crucial: set the gain too low and weak targets disappear; set it too high and the display saturates with background noise or sea clutter. On older or lower-end radars, the gain is adjusted manually by the operator; typically increasing it until noise speckles just become visible, then dialing back slightly. Skilled operators constantly tweak the gain to maximize target visibility without overwhelming clutter. In heavy clutter (e.g. high sea state or rain), an operator may reduce gain to avoid false returns, at the cost of possibly losing very small echoes. A simple example of high and medium clutter environments is shown in Figure 22. As can be seen in Figure 22, although a gain setting can work in one environment, it does not necessarily guarantee it will work in another environment. Thus the major drawback of manual tuning is its subjectivity and inclination towards continuous readjustment.

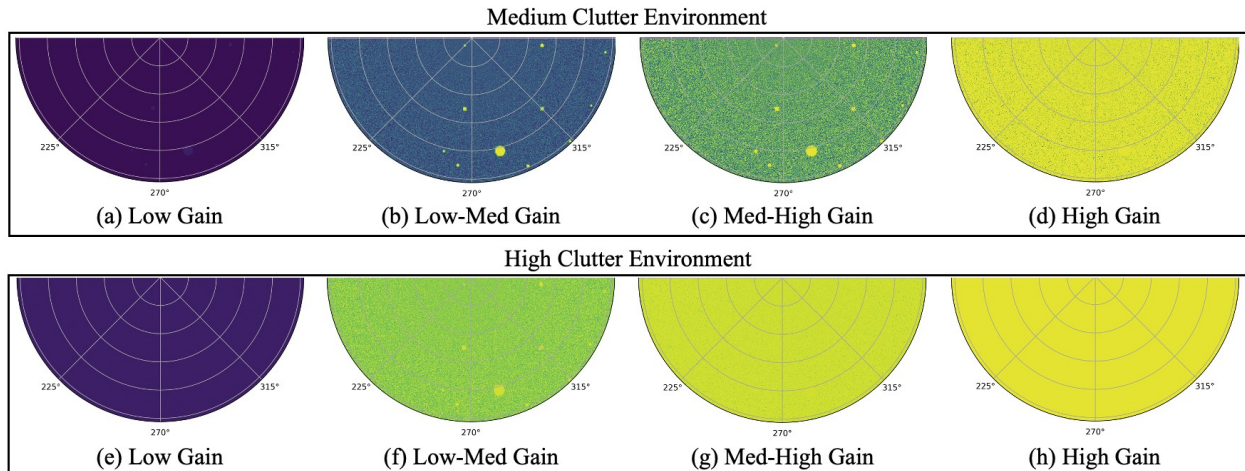


Figure 22: High and Medium Clutter Environments with Varying Gain Levels

Modern radar systems (such as the commercial ones utilized for this project) often employ Automatic Gain Control (AGC) to continually adapt receiver gain based on overall return levels. AGC will lower gain when the radar sees a lot of strong returns (for instance, large waves or nearby land) to prevent the display from washing out, and raise gain in calmer conditions to maintain sensitivity. This adaptive approach keeps the output within an optimal range without operator intervention. However, AGC is a blunt instrument; it does not discriminate between wanted targets and unwanted clutter. In the presence of heavy clutter, AGC may inadvertently suppress small target echoes along with the background, since it simply equalizes the total output level. Thus, a tiny kayak could be damped out when large waves are in the vicinity, as the radar autonomously reduces gain to handle the clutter. Finding the right balance is difficult, which is one reason we are motivated to handle raw radar data in a more intelligent way (using sensor fusion and learning, as described later) rather than relying purely on built-in AGC.

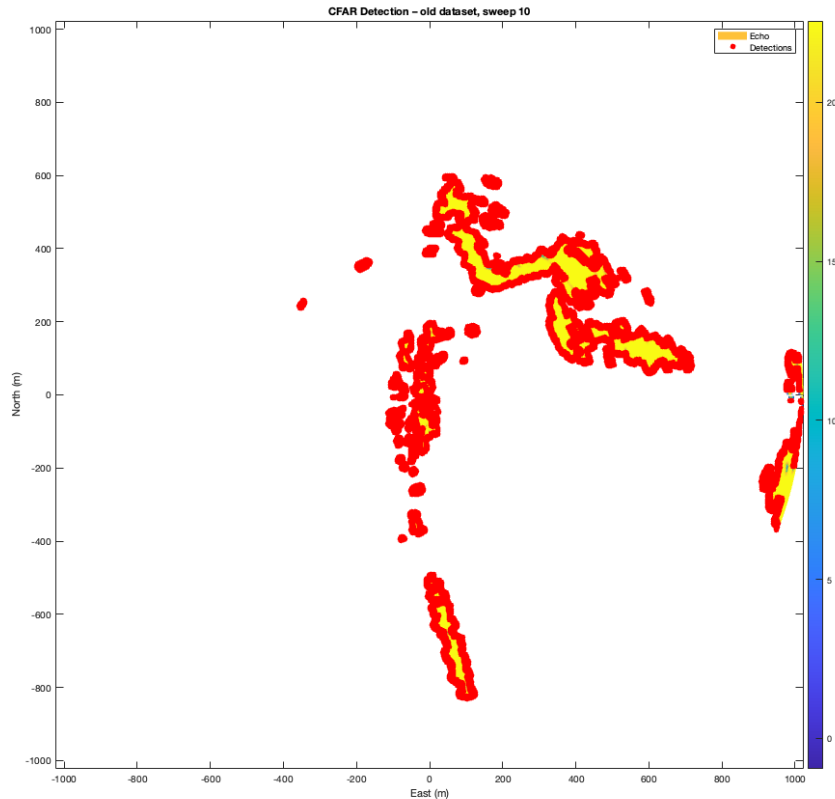


Figure 23: CFAR Implementation for Data Collected at Lake Murray

Another fundamental radar processing technique is the Constant False Alarm Rate (CFAR) detector. The elementary basics of CFAR is shown graphically in Figure 24 while a version of CA-CFAR is shown on real data in Figure 23. CFAR adaptively sets the detection threshold based on local clutter statistics to maintain a roughly constant false alarm probability. In practice, the radar computes the average or distribution of the return power in a sliding window around a cell and chooses a threshold such that, say, only one in a million noise cells exceeds it (for a false alarm rate of 10^{-6}). Any echo stronger than this threshold is declared a target. CFAR is very useful in maritime radar since sea clutter is not uniform: it raises thresholds in regions of heavy backscatter (e.g. breaking waves) and lowers thresholds in calmer areas. This adaptivity prevents a flood of false alarms that a fixed threshold would trigger in rough seas, and conversely, it retains sensitivity in calm waters where a fixed high threshold would miss targets.

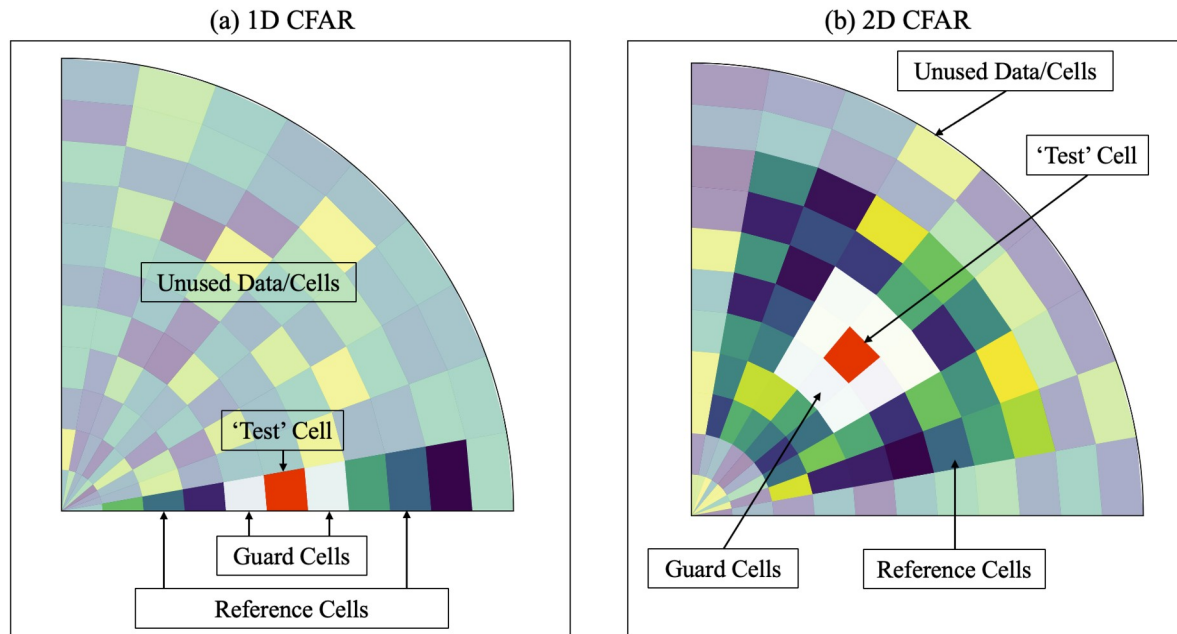


Figure 24: CFAR 1D and 2D on Sample Radar Data

In addition to requiring tuning for each radar, CFAR is not foolproof. In intense, non-homogeneous clutter, a small target's echo may be submerged below a heightened local threshold; the radar effectively raises the bar so high (to keep false alarms down) that the target is missed. For example, a slow-moving person in water surrounded by choppy waves might be invisible to a standard CFAR detector because the local sea clutter drives the threshold above the person's return level. On the flip side, in non-homogenous clutter scenarios, CFAR can become overly sensitive, requiring careful tuning of the false-alarm rate to avoid peppering the screen with false targets. These limitations motivate exploring machine learning approaches, which can potentially learn to recognize the subtle signal of small objects within clutter and motivate using additional sensor modalities (camera) to confirm which radar returns are real targets.

Part 2.3 Planned Sensor Fusion Approach

By combining radar and camera data, it is believed there is a path to capitalize on the strengths of each. The radar provides range, bearing, and a detection confidence (echo amplitude) for objects, even in darkness or haze. The camera provides rich appearance information to classify and localize those objects in image space. A critical step in fusion is sensor co-registration; i.e. calibrating the coordinate frames of the radar and camera so that we can map a radar detection (range θ , bearing ϕ) to the corresponding pixel location in the camera image. A calibration technique has not been settled upon yet, but will likely be of the autonomous variety.

Once this calibration is in place, each radar detection can be projected into the camera view as an annotated bounding box or marker. This allows a few immediate benefits: (1) Improved detection - if the radar detects something that the camera-based algorithm missed, the system can direct attention to that region in the image (for a potential false-negative recovery); and (2) Assisted labeling - as we begin building a training dataset for camera object detection, the radar can pre-suggest regions of interest, drastically reducing the human effort needed to scan each frame for tiny objects. In essence, the radar acts as an "extra pair of eyes" for the camera, especially in scenarios of poor visibility or tiny targets. The aim is to implement a radar-assisted labeling pipeline where radar detections provide initial bounding boxes for objects in the camera images. A human annotator (or an automatic verification step) would then confirm or adjust these boxes,

resulting in labeled images with significantly less manual labor. A simple diagram with real sensor data overlaid by a possible implementation for human annotators is shown in Figure 25. This approach addresses the aforementioned labeling bottleneck by focusing human attention on likely object locations instead of erratically and aimlessly searching large empty frames for small objects.

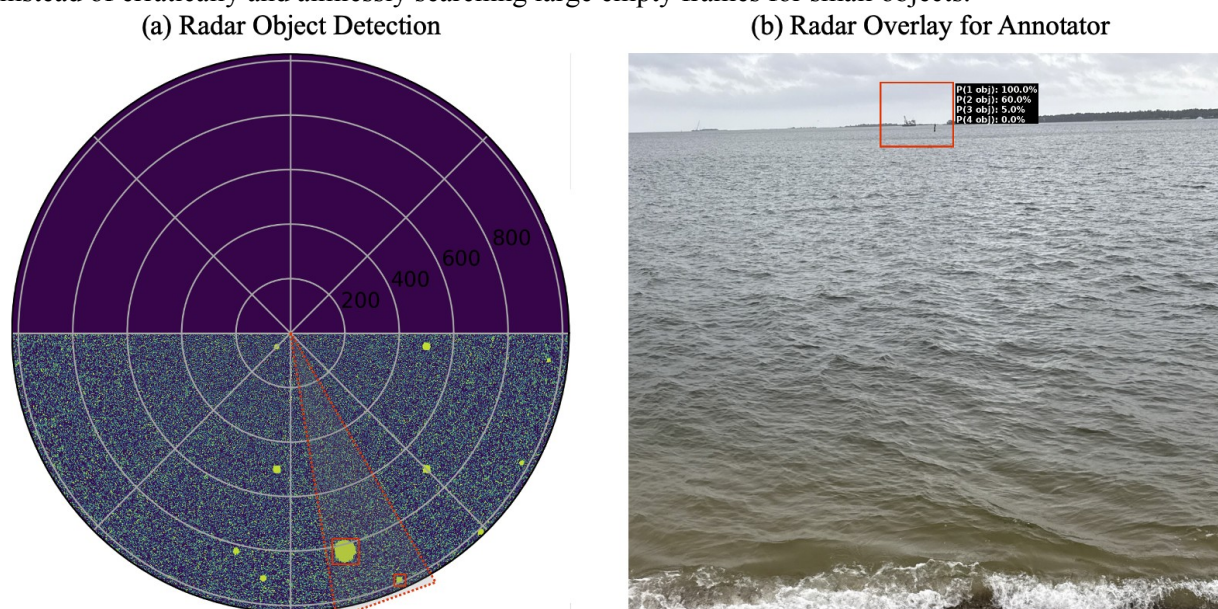


Figure 25: Annotator-Radar Labelling Pipeline

Part 2.4 Multi-Gain Radar Data for CNN

A novel aspect of the approach is how Radar gain is handled. As discussed, a single fixed gain (or even AGC) can either wash out weak targets or flood the display with clutter. Our solution posits to capture multiple radar intensity frames at different gain levels and feed them together into a neural network, allowing the network to learn a composite understanding of the scene. The marine radar we use can be configured to produce such multi-gain outputs via low-level manipulation of the radar software, and utilizing temporal-based techniques. By doing so, the chosen detection architecture can avoid committing to a single threshold early on. If the gain is too low, one frame will show almost nothing – even strong returns like land might barely appear. Conversely, if the gain is too high, another frame will show everything lit up (strong targets and noise speckle everywhere) so that true targets are hard to distinguish. In practice today, an operator must manually adjust gain to find a Goldilocks setting for each situation. To reduce this dependence on an expert user, we propose an automatic multi-gain fusion. Because the radar display’s native output tends to be binary (see Figure 26) at any given gain, effectively an aggressive thresholding that yields mostly 0 or 1 in the pixel matrix, no single setting can capture all targets. By combining multiple “exposures” of the radar at low, medium, and high gain, it is possible to obtain a richer representation of the underlying signal strength.

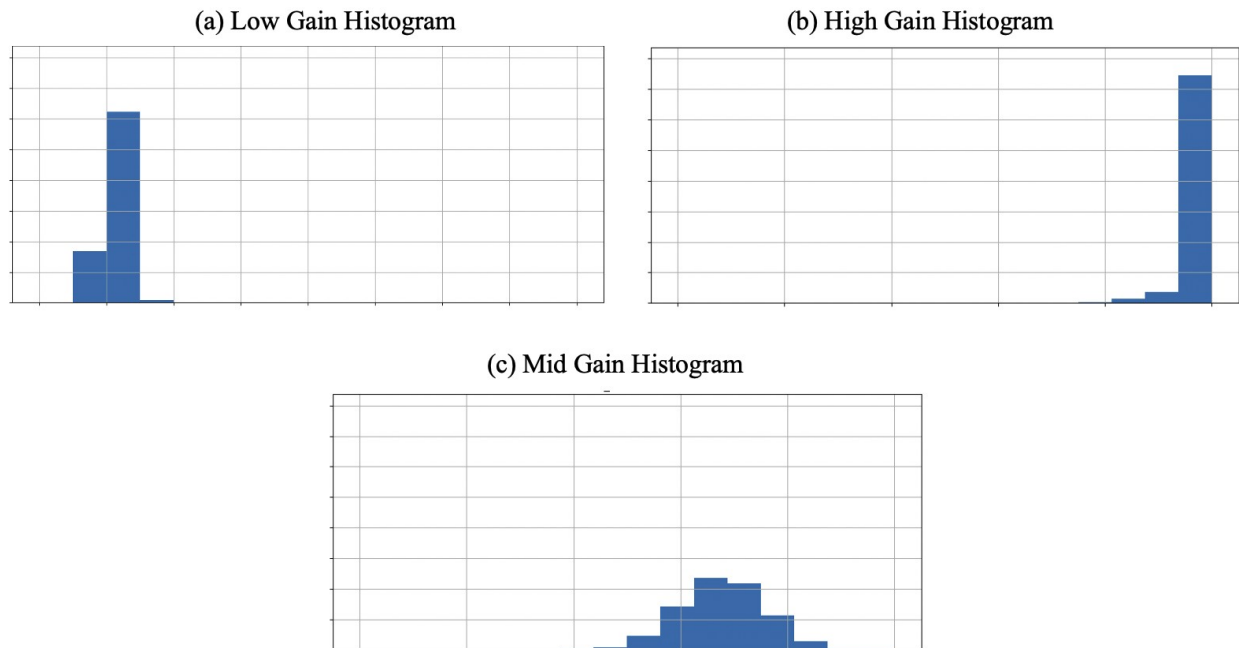


Figure 26: Histograms of Different Gain Settings – 4-bit Quantized

The neural network will receive a stack of these frames (e.g. as separate input channels), from which it can deduce that an object which is invisible in the low-gain frame but pops out in the high-gain frame is likely a small target in heavy clutter, whereas something visible in low gain but blurred in high gain might be a strong target or interference. Feeding multiple gain levels into a CNN is expected to improve small object detection, essentially providing the network with dynamic range context that the radar’s own display thresholding throws away. This idea is inspired by the concept of multi-exposure imaging in photography, where combining images taken at different exposures can reveal details in both dark and bright regions. In our case, the “exposures” are radar gains. An initial convolutional model (e.g. a custom ConvNet or an adaptation of YOLO for radar) will be developed to take these multi-gain radar frames and output detections (range, angle) of potential targets. Those detections can then be mapped onto the camera images as described. Ultimately, the output of the fusion system will be both a radar PPI with target markers and camera frames with annotated objects; a comprehensive situational picture. Preliminary results for gain-combining and clutter removal under High clutter simulated settings is shown in Figure 27 with somewhat promising results.

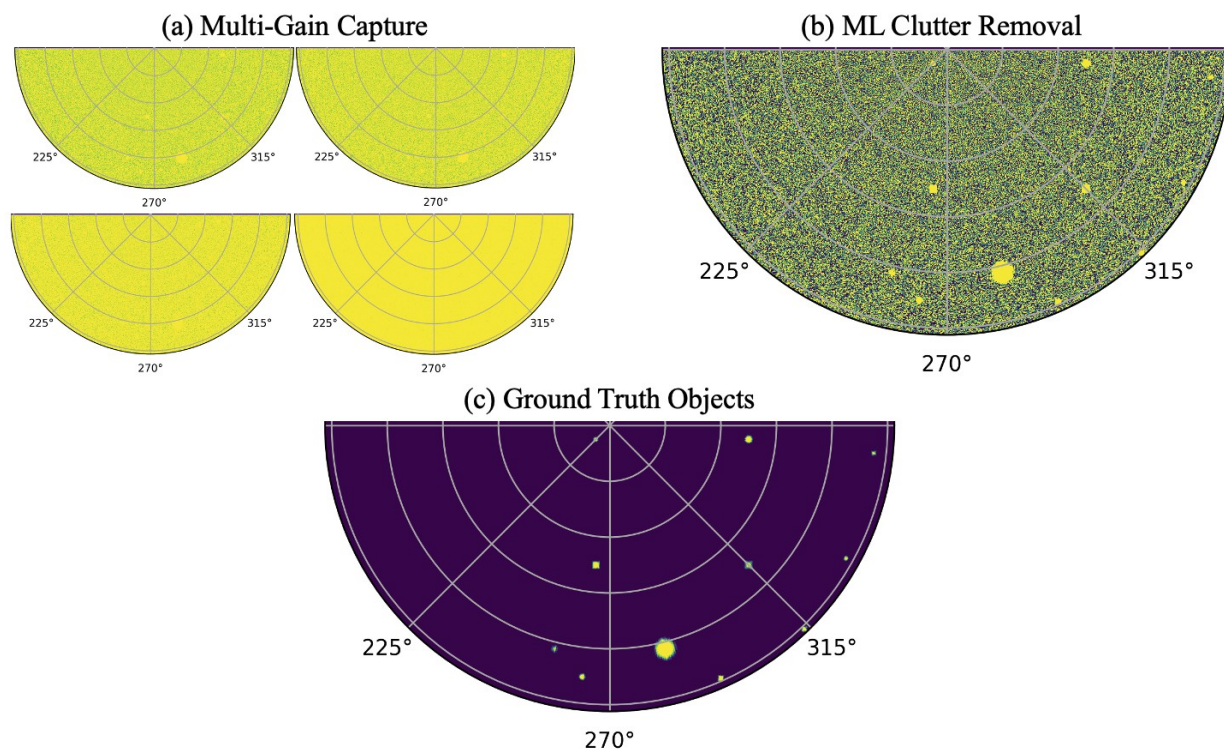


Figure 27: Multi-Gain Radar Detection for Stationary Low-RCS Objects in High Clutter Environments

Part 3 Planned Work

Over the next several months, the plan is to design and implement an end-to-end radar-camera object detection pipeline that demonstrates the advantages of fusion and multi-gain radar processing. The major components and milestones of this research ‘phase’ are as follow below.

Part 3.1 Build the Sensor Rig

Set up the experimental platform combining a marine radar and an optical camera (and optionally an AIS receiver) on a test vessel or shoreline station. This includes hardware integration and calibrating the relative orientation and position of the radar and camera (i.e. performing the co-registration so that we can accurately overlay radar detections on camera images). Timing synchronization between sensors will be completed to ensure each radar sweep can be paired with the corresponding video frame. The current testing rig and a sketch of a proposed new testing rig is shown in Figure 28.

(a) Current Collection System



(b) Proposed EZ-Setup System



Figure 28: Radar Data Collection System - Current & Planned

Part 3.2 Collect Pilot Dataset

Conduct data collection trials to gather a pilot dataset of synchronized radar and camera recordings. The aim is to collect on less than 10 hours of radar PPI video (scans) with corresponding camera footage in various conditions. These will include scenarios with known small targets (e.g. a kayak, buoy, pylons) as well as background-only runs. Importantly, we will record raw radar data without lossy compression. The pilot dataset will be curated as A-MARV-pilot, a preliminary version of what will become a larger Amplitude-only Maritime Radar Video (A-MARV) dataset. All data will undergo minimal preprocessing: filtering obvious noise, applying our multi-gain transformation on each radar frame, and storing aligned radar-camera pairs for training and analysis. The initial data is planned to be collected at Lake Murray with future locations to include SC lakes and shorelines before moving to more complex and diverse sea scenarios. The full selection of lakes planned for the initial release of lake data is shown in Figure 29.

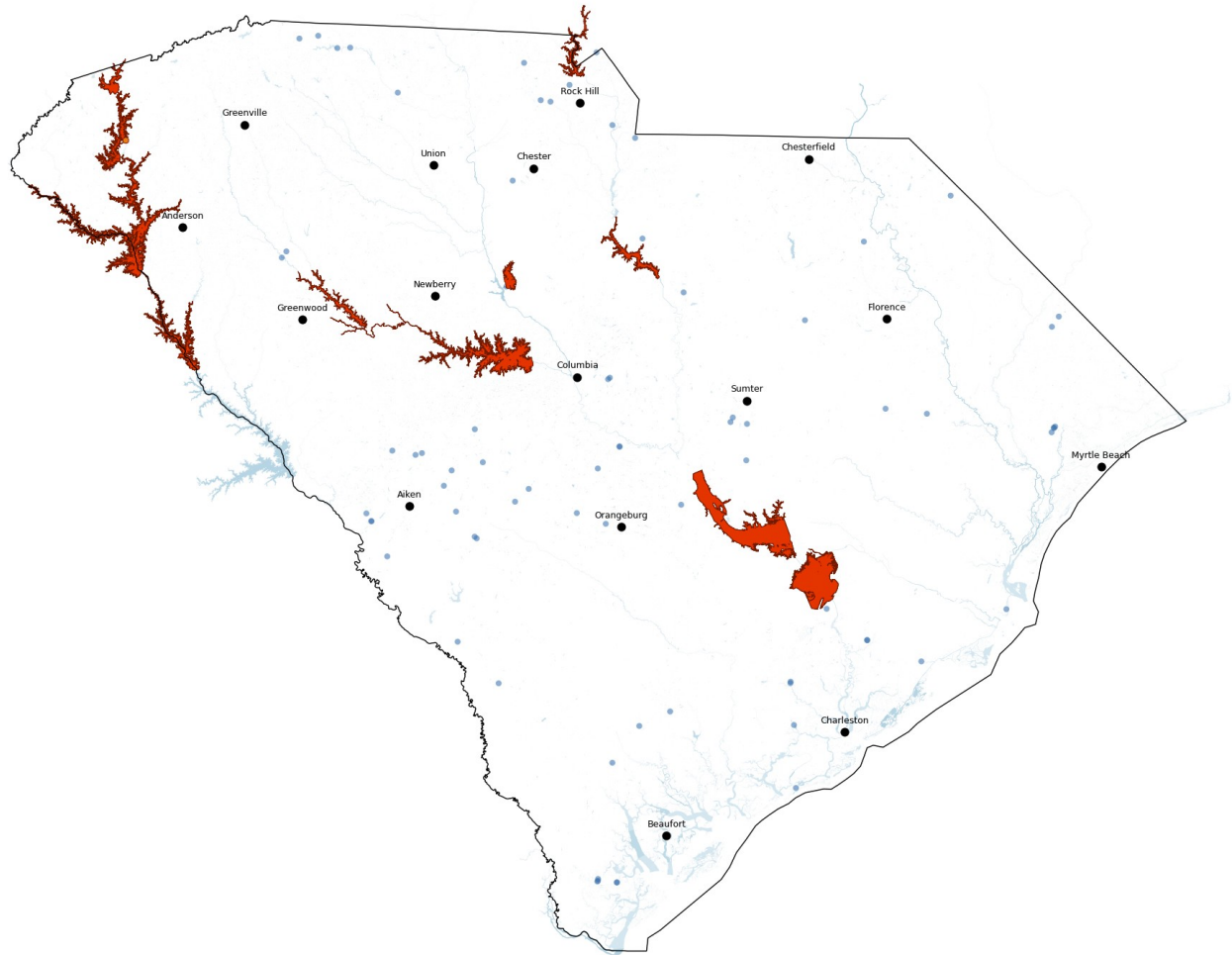


Figure 29: Planned Lakes for Data Collection

Part 3.3 Multi-Gain Data Fusion Technique

Implement the scheme of generating multi-gain radar input frames. We will produce, for example, three intensity images: one with low gain (conservative threshold), one medium, and one high gain (sensitive). These three images form a multi-channel radar representation of that moment. This step may involve developing a script or using the radar's SDK to adjust gain or threshold levels. We will validate that this approach indeed highlights targets across the frames, e.g. a small kayak might only appear in the high-gain image, whereas land or large vessels appear even in low-gain. The combination effectively spans the dynamic range of returns.

Part 3.4 CNN Model Development

Design and train a baseline convolutional neural network that ingests the multi-gain radar data (and potentially temporal sequences) to detect objects. As a starting point, we plan to implement a 1 D convolutional detector that scans each radar range-azimuth slice (similar to a Conv1D over the radar's polar coordinate image). This simple model will be trained to distinguish target returns from clutter using the pilot data. We will compare its performance to the classical 1D CA-CFAR detector as a baseline. The hypothesis is that even a basic learning-based model can achieve better sensitivity than CFAR for low-RCS targets, when provided with multi-gain inputs. As we iterate, more sophisticated models (e.g. 2 D CNN on the Cartesian radar image or even a radar-camera fusion network) may be explored, but the initial focus is on getting a working radar-target detector.

Part 3.5 End-to-End Detection Pipeline

Integrate the radar-CNN and camera. Detected target coordinates from the radar-CNN will be mapped onto the camera frame in real-time (or offline, initially). We will implement a simple user interface to visualize results – for example, drawing radar detections as bounding boxes on the camera video. This will serve as a radar-assisted labeling tool: when we pause the video, the system can output suggested annotations for any objects the radar picked up. The pipeline will be evaluated on the pilot dataset by measuring detection rates and correctness of the suggested labels.

Part 3.6 Preliminary Evaluation and Iteration

Evaluate the performance of the radar-based detector versus traditional methods. Specifically, we will compare detection outcomes of our CNN to the radar’s standard ARPA implementation. Key metrics include probability of detection for the small test targets and false alarm rates. If the initial model underperforms in some scenarios (e.g. very rough clutter), we will analyze the failure cases and consider additional inputs or model architecture changes. We expect to demonstrate improvement over CFAR, at least in moderate conditions, as a proof-of-concept result suitable for publication.

Part 3.7 Radar-Assisted Image Labeling

As an offshoot of the detection pipeline, develop a semi-automatic labeling tool for the camera imagery. This tool will use the radar detections (after filtering out obvious false alarms) to mark regions in each video frame that likely contain an object. A human annotator can then quickly confirm the presence of an object and label its bounding box or class, rather than searching from scratch. We will apply this process to our pilot dataset to generate training labels for the camera. This not only accelerates data annotation but also yields a more diverse training set of small-object images than could reasonably be labeled manually. The effectiveness of radar cueing will be quantified by measuring the reduction in manual effort (e.g. how many proposals per hour the radar provides and what fraction are true objects). This addresses the objective of an automated labeling pipeline using radar cues.

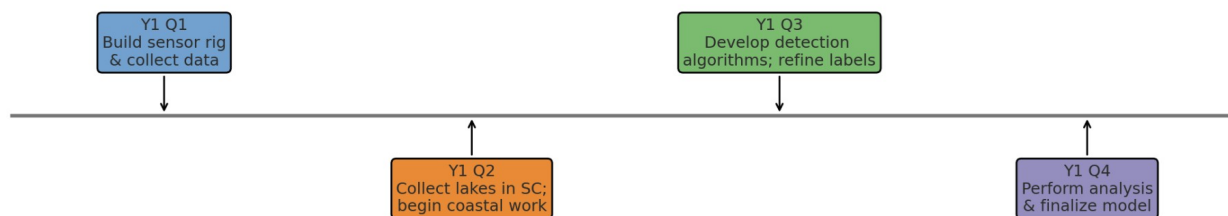


Figure 30: One-Year Timeline of Project

Part 4 Later Work

This first phase of research will establish a strong foundation for a longer-term Ph.D. roadmap. In the future, we plan to build on the prototype and concepts developed during the Master’s work to tackle deeper challenges in radar perception and sensor fusion. Some future research directions include:

Part 4.1 Self-Supervised Radar Representation Learning

I will explore training the radar detection models with self-supervised learning techniques to reduce reliance on extensive labeled data. One planned approach is a Radar Masked Autoencoder (Radar-MAE), where the model learns to reconstruct missing portions of radar scans, thereby learning robust features from raw data. By pre-training on a large amount of unlabeled radar video (tens or hundreds of hours), the model can develop an understanding of sea clutter characteristics and typical target signatures. These features can then be fine-tuned for detection tasks with relatively few labeled examples (few-shot learning). This thrust is expected to lead to a conference publication (e.g. CVPR ’27 or ’28) showcasing that a self-supervised radar model outperforms a fully supervised one given limited labels.

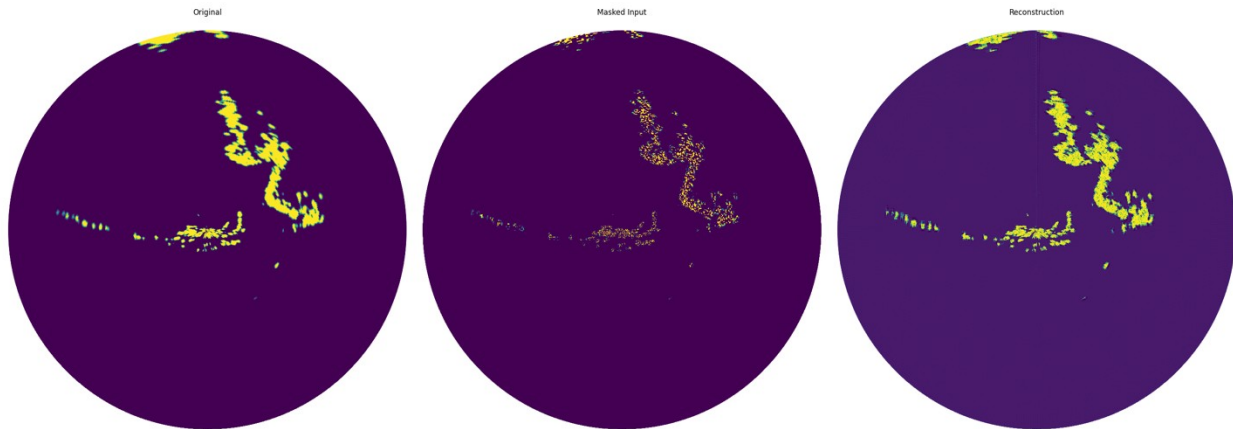


Figure 31: Results of preliminary MAE training (<10 minutes on MacBook Air)

Part 4.2 Physics-Informed and Continual Learning

Another future direction is to incorporate radar signal processing physics into the neural network architecture. For example, embedding the radar’s known range equation or CFAR principles as differentiable components (loss terms or network layers) could guide the model to make physically plausible detections (controlling false alarm rates explicitly). We will also investigate Vision Transformer (ViT) models tailored to radar by encoding priors like the pulse repetition frequency and antenna patterns. In tandem, as we deploy the system in varied environments, continual learning strategies will be needed so the model can adapt to new sea states or sensor configurations without forgetting old knowledge. Techniques like online domain adaptation and Elastic Weight Consolidation (EWC) are planned to ensure the radar perception system remains robust over time. These advanced topics form the core of much later work, aiming for journal-level contributions in the radar/remote sensing field.

Part 4.3 A-MARV Dataset Expansion

Throughout the entire project, we expect to significantly scale up our data collection to build the Amplitude-only Maritime Radar Video (A-MARV) dataset; the first open benchmark dataset of its kind. The full A-MARV will comprise many hours of real radar recordings (X-band, ~9.4 GHz) captured in diverse conditions, using a tri-gain recording strategy (temporal low/medium/high gain frames) to preserve small-target visibility. It is planned to label a portion of this data with “strong” annotations (definite object labels) and an even larger portion with “weak” annotations generated by semi-automatic means (e.g. the radar-assisted labeling plus a data programming approach using tools like Snorkel). The A-MARV dataset will be published with standardized tasks (detection, domain adaptation, continual learning, few-shot learning) to catalyze research in maritime radar ML. The goal is to eventually release A-MARV along with baseline models and an evaluation toolkit, establishing a community resource. This will not only validate the broader impact of our work but also ensure reproducibility and benchmarking for future researchers. A dedicated data descriptor paper (e.g. in IEEE T-GRS) is planned once the initial version of the dataset is ready.

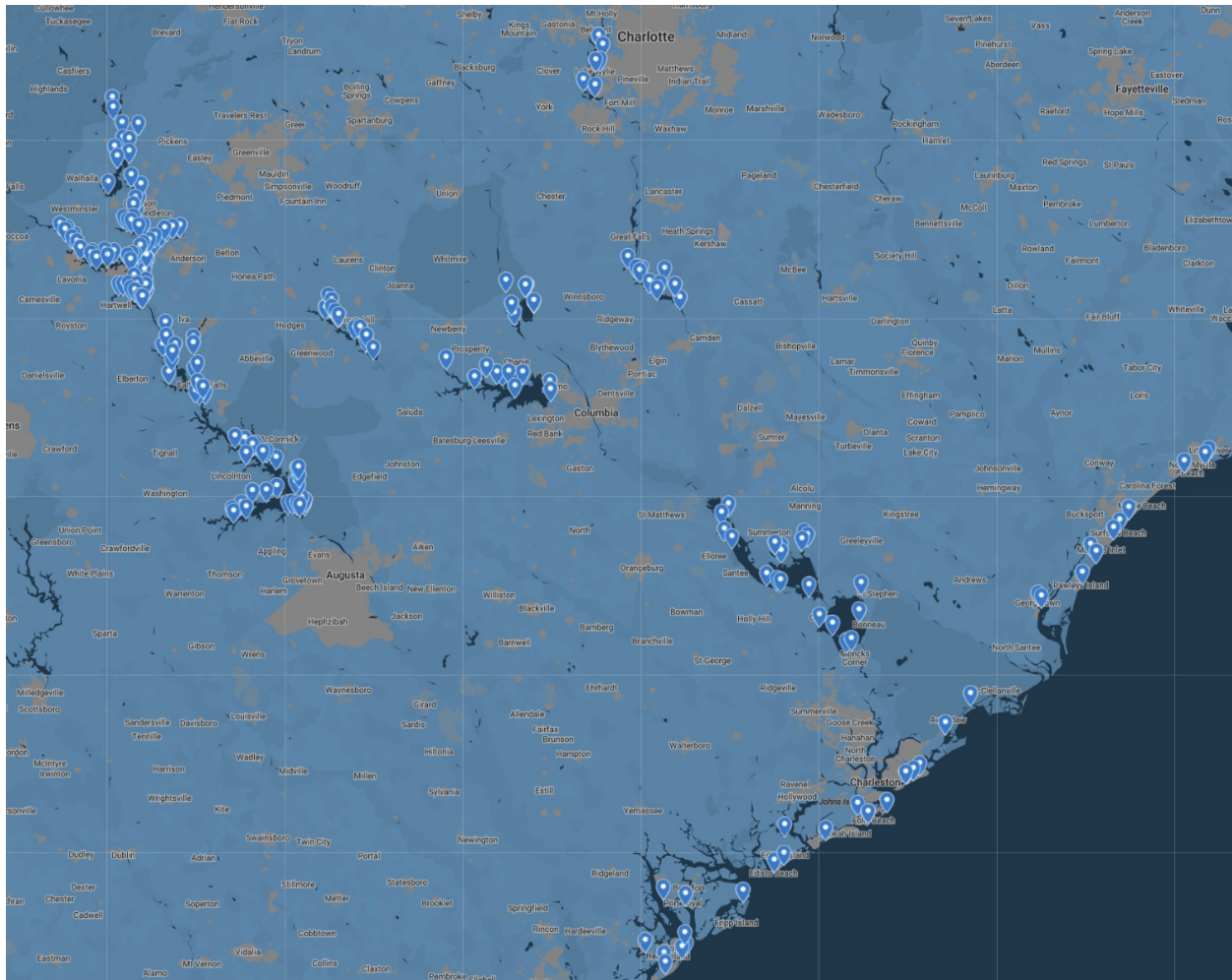


Figure 32: Initial (v0.1) A-MARV Dataset Collection Locations

So far, the core prototype has been established, integrating a multi-gain radar with high-resolution imagery and training initial detection models, and run our first small-object detection trials. Over the coming months, we'll focus on quantifying gains in small-object detection and automated labeling accuracy as well as sharing performance benchmarks and system design insights

Part 5 Expected Results & Outcomes

Improving the detection of small objects in the maritime domain has significant real-world implications. From a maritime safety perspective, the ability to spot a person overboard or a semi-submerged hazard (e.g. a floating log or container) can save lives and prevent collisions. Small vessels such as kayaks or dinghies are often missed by large ship radars until dangerously close; a better detection system could alert operators or autonomous navigation systems earlier.

Enhancing radar detectability of these objects – without overwhelming false alarms – fills a critical gap in surveillance of ports, coastal waters, and open oceans. This is increasingly important as we move toward unmanned surface vessels and advanced maritime traffic monitoring, where reliable automation of small object detection is required for safety and security. The fusion approach in this project also offers a valuable benefit for data generation and algorithm development.

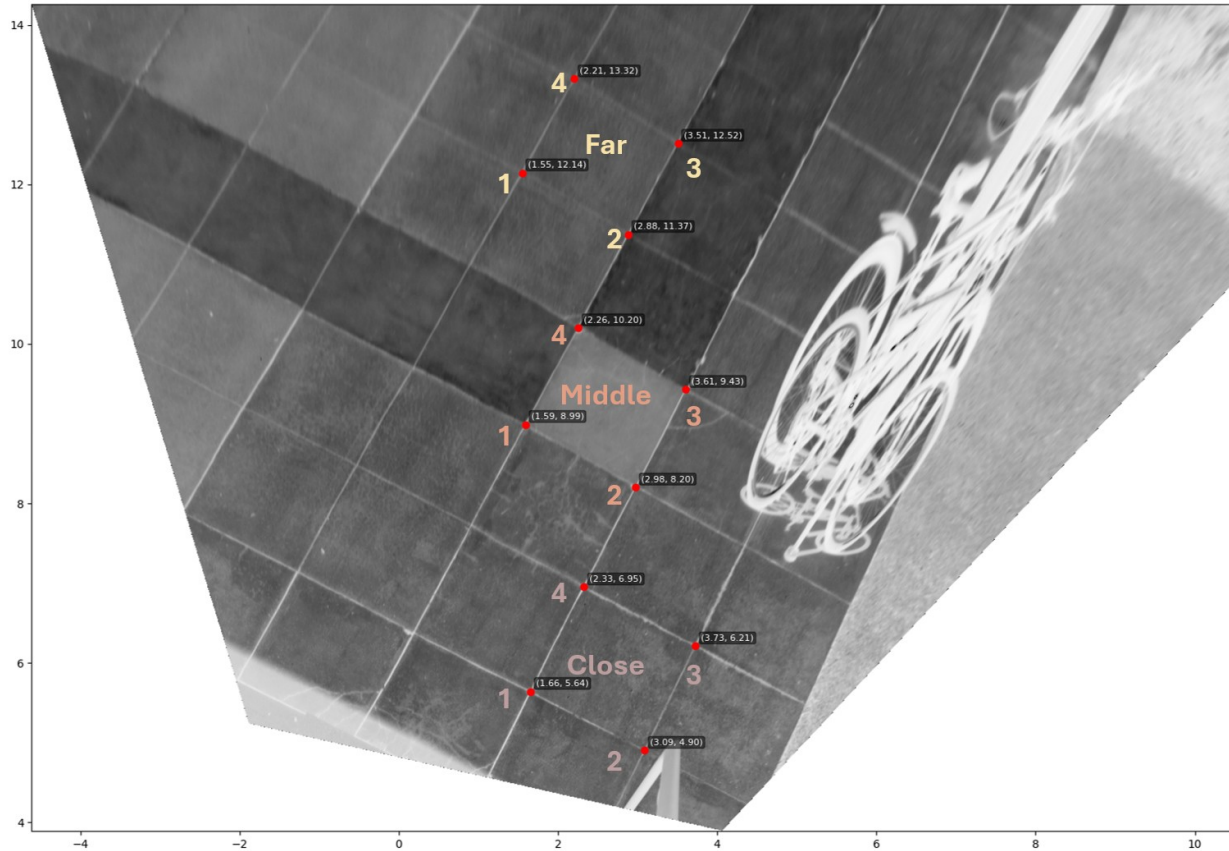
By using radar to assist in labeling optical imagery, we tackle the data scarcity issue that plagues maritime computer vision. Instead of relying on hundreds of hours of tedious manual annotation to train a camera-based detector, the radar's cues allow us to rapidly label a high-quality dataset. This radar-assisted labeling not only reduces human workload dramatically, but it also improves dataset quality; the labels are more

likely to catch hard-to-see objects that a human might overlook when fatigued. In essence, our approach leverages the complementary nature of the sensors: radar provides precision in detection cueing, and the camera provides rich detail for classification. Together, they yield a system more robust than either alone, particularly for the challenging maritime environment.

2.2.3 Orthorectification Testing

Orthorectification was evaluated during this period using images captured in front of the Swearingen Engineering Building. The ground in this area has standardized concrete tiles measuring approximately 1.524 meters by 1.524 meters, making it an ideal reference surface for assessing the accuracy of the rectified images.

Manually measured corner points for three specific tiles; located near, mid-range, and far from the camera; were used to calculate edge lengths, as shown in Figure 33. The resulting side lengths from the rectified images are listed in Table 1. These measurements show variations along the tile edges, indicating some error in the rectification process. This level of error was expected due to the use of human measurements for camera translation. Future work will incorporate direct INS measurements to improve the accuracy of camera translation data over time.



Error percentages for each edge of the near, middle, and far tiles are shown in Figure 34a. From this figure, it is evident that errors in the x-direction increase with the distance of the tile from the camera. This trend suggests that camera roll contributes significantly to error in the x-dimension. Figure 34b presents the average distance error for each tile. While the magnitude of error increases with distance from the camera, all errors remain below 10%, which is a promising result given that most measurements were obtained manually.

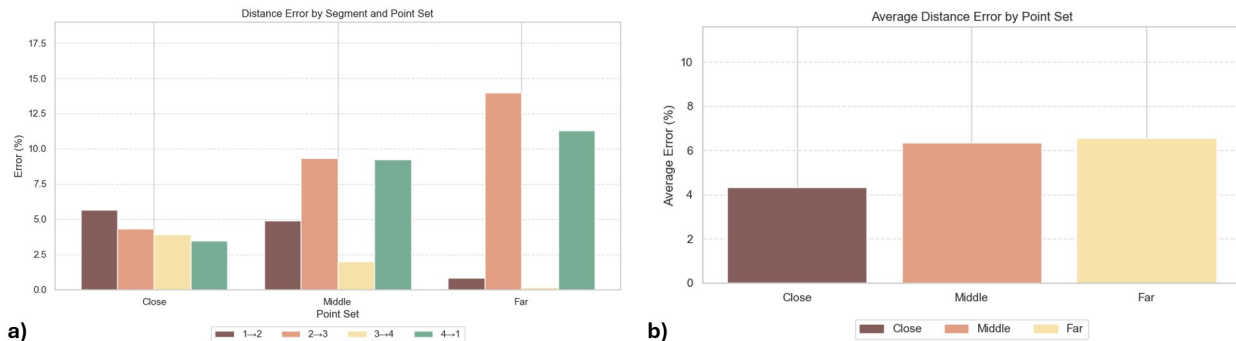


Figure 34: (a) side length error for each case and (b) average error between side lengths for each case

Distance Cutoff

A major improvement in the orthorectification pipeline during this period involved switching from a fixed image point cropping method to one based on a world coordinate distance cutoff. This change ensures that rectification does not attempt to map pixels that are projected far from the image plane like points near the horizon. Before, this would lead to rectification “to infinity” giving memory allocation errors, but this fixes this problem without having to evaluate every image for the best distance to cut the images.

The process works by iterating along the edge pixels of the undistorted image and calculating the world coordinates of each pixel until the world coordinate value is greater than the desired cutoff distance. As the pitch angle increases, the number of pixels cut off the image is less. Once the pixel at the given distance along the edge is determined it is used as one of the 4 corner points for image warping. Using these points, the image is warped to world coordinates. This process is shown in Figure 35.

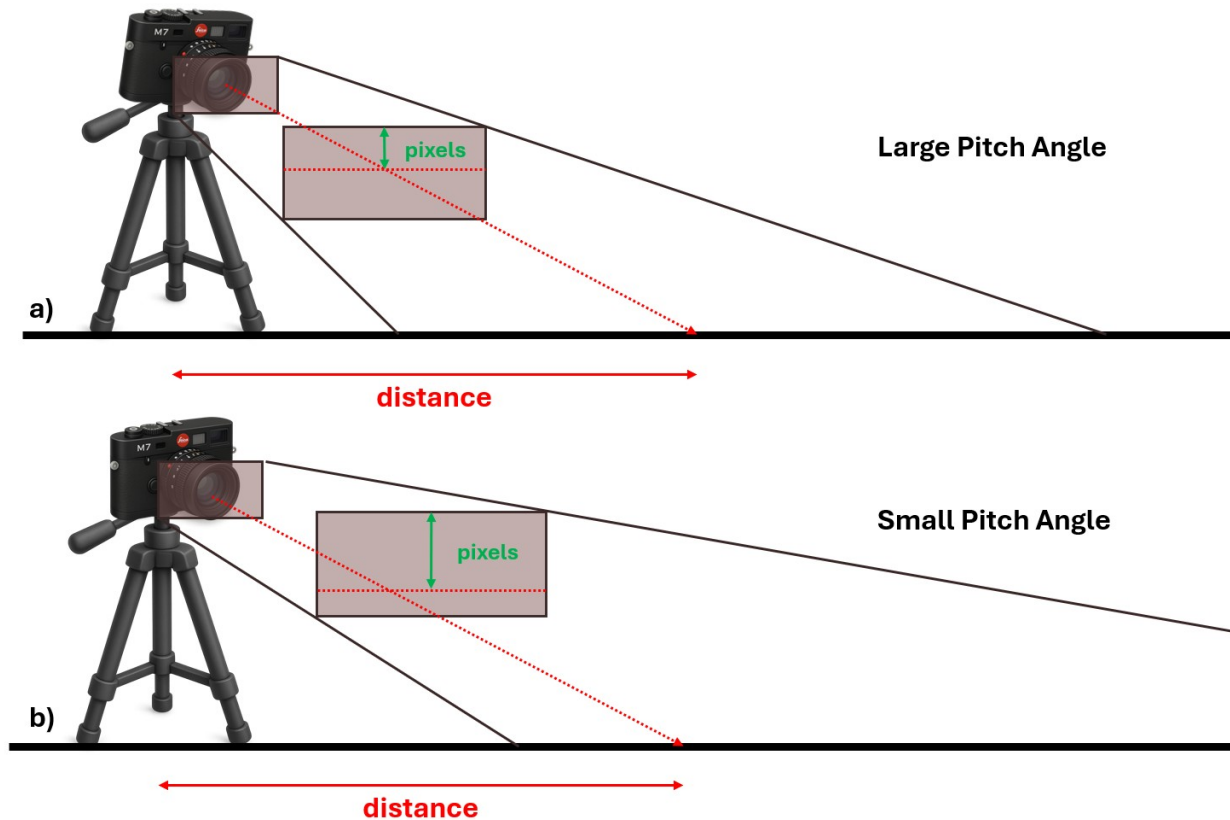


Figure 35: Diagram showing how images are being cut to world distance

2.2.4 Radar-Camera-INS Integration on Jetson AGX Orin

Part 1: Introduction

During this work period, the JetSeaAI SIMRAD Halo24 SDK and its ROS2 driver were enhanced on the Jetson AGX Orin to enable persistent recording of raw radar data. The purpose of producing timestamped radar logs is to align each 360-degree sweep with camera imagery and INS metadata for downstream sensor fusion analysis. To achieve this, the SDK was modified to intercept the User Datagram Protocol (UDP) sector packets as soon as the SIMRAD initializes. The ROS2 node decodes each packet into a discrete spoke, comprised of 1024 echo values at 0.175781-degree resolution. These spokes are buffered until 2048 of them (one full 360-degree rotation) have been assembled. Upon completion of each rotation, the node saves the buffer to the Orin as a txt file. Each file contains microsecond-precision timestamps, sweep length, azimuth angle, GUI-configured range scale, and all echo intensity values. Files are organized in uniquely named, timestamped directories on the Orin. Please see Figure 36 for the radar data saving workflow. This workflow provides lossless, rotation-aligned files that ensure precise time synchronization between radar, camera, and INS data.

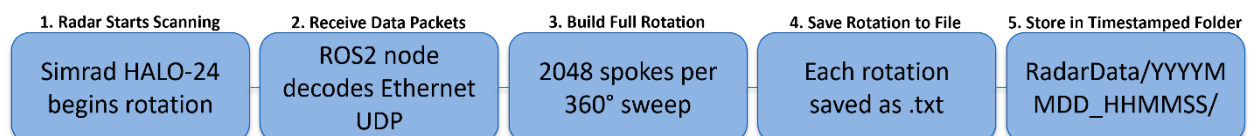


Figure 36: Raw Radar Data Saving Pipeline

Each full 360-degree rotation is captured in a txt file stored on the Jetson AGX Orin under a uniquely named, timestamped directory. Directories are named by the recording start time (e.g., 20250606_125516) and each file follows the convention: “Radar_raw_data_YYYYMMDD_HHMMSS.sss_N.txt”. Where “sss” is milliseconds and “N” is the rotation index. This makes it easier to correlate with camera frames and INS logs. See Figure 37 for example saved radar data.

In each file, every spoke includes the following fields:

- **Timestamp:** Microsecond-precision time when the spoke arrived.
- **Sweep length:** Number of echo samples in the spoke (fixed at 1024 values).
- **Azimuth angle:** Bearing of that spoke (0.175781-degree resolution, covering 0-360-degree).
- **Range scale:** Maximum detection range in millimeters as set in the GUI.
- **Echo intensities:** 1024 values (0-15) indicating return strength for each sample.

Due to the file’s use of plain text, they can be opened in any editor then converted to “.mat” files with ease for visualization and sensor fusion. The saving format follows the official SDK structure, allowing compatibility with existing MATLAB postprocessing scripts.

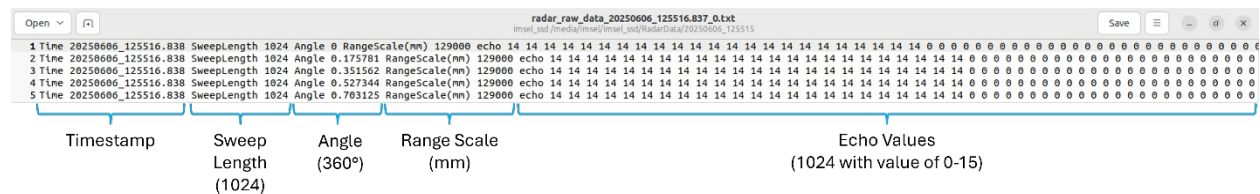


Figure 37: Saved Fields of Raw Radar Data

Part 2: Fused Radar-Camera System

With the Orin now saving raw radar data, the complete radar-camera-INS platform captures every stream with precise timestamps for downstream visualization and fusion. Figure 38 shows the hardware layout and data inputs. SIMRAD streams 360-degree sweeps over standard ethernet. The Lucid camera delivers 5 frames per second (FPS) raw images via M12 ethernet. Finally, SBG INS provides 5 Hz inertial and GNSS data over USB and issues trigger pulses to the camera via GPIO. Power is supplied to all devices via 12V supply. Each sensor writes its output to a dedicated, timestamped directory on the Orin’s SSD, making it easy to align radar text logs, camera frames, and INS files during postprocessing.

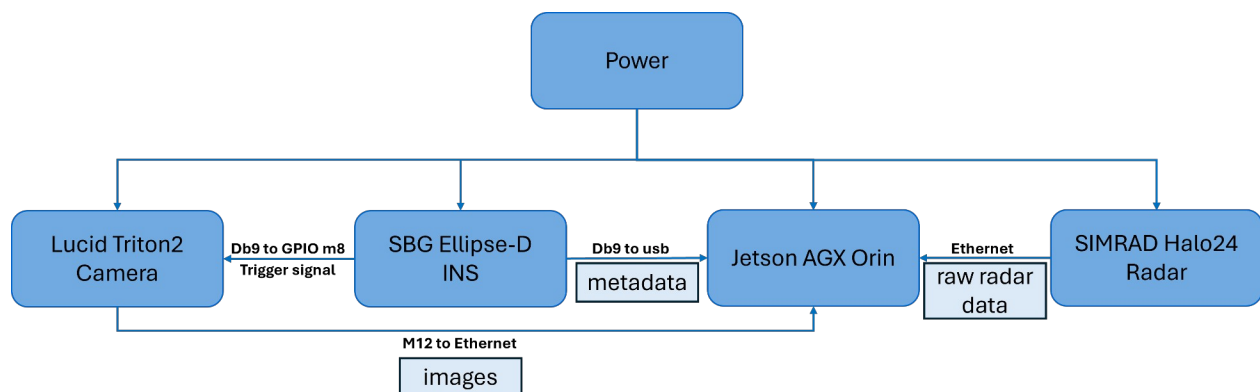


Figure 38: Fused Sensor System Wiring Diagram and Data Inputs

Prior to data collection, the operation was validated by launching the radar and camera/INS ROS2 nodes together and observing real-time outputs in a three-panel interface (Figure 39). In green, the SIMRAD Halo24 GUI displays scan parameters (range, gain, etc.) and system health, while allowing live adjustments. In red, the visualization node renders each decoded spoke onto a polar plot confirming 360-

degree coverage and echo intensity patterns as the radar spins. In blue, the Lucid camera's live feed streams at 5 FPS, each triggered by a GPIO pulse from the SBG INS to guarantee sync frames. Throughout the test, all three streams write simultaneously to timestamped directories on the Orin's SSD, ensuring time alignment for downstream fusion processing.

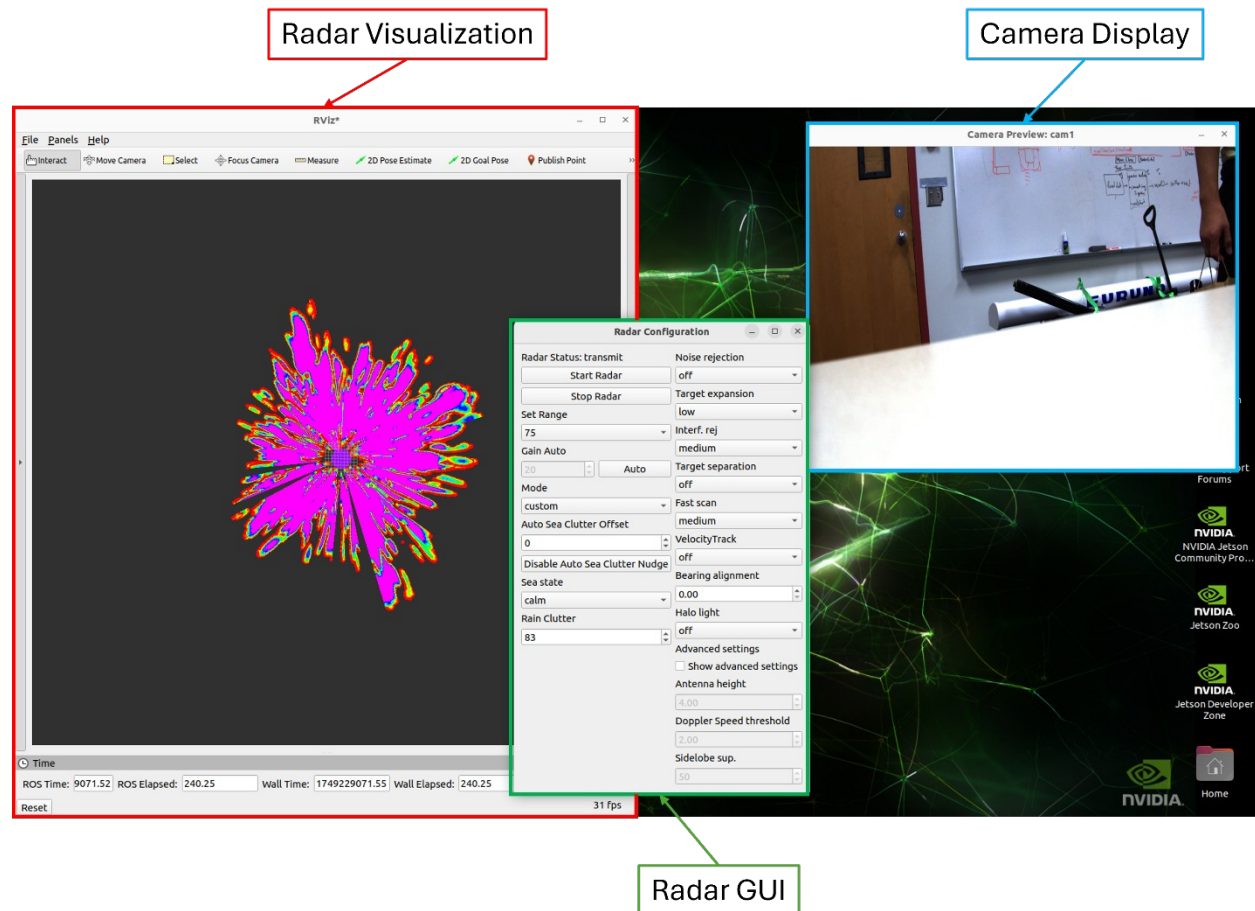


Figure 39: Fused Sensor System Visualization and GUI

After confirming synchronized operation during runtime, the saved data was inspected to ensure proper file structure and organization. As shown in Figure 40, the radar, camera, and INS outputs are each stored in their respective timestamped directories on the Orin's SSD. Radar rotations are saved as sequential ".txt" files. Camera frames as ".raw" images, and INS metadata as individual ".txt" files per frame. This consistent and clearly partitioned folder structure allows for straightforward batch loading and time aligned fusion in postprocessing workflows.



Figure 40: Simultaneous Data Saving of Radar, Camera, and INS

2.2.5 Rain Segmentation Model Improvements

Part 1: Training a rain segmentation model

We trained a rain segmentation model based on the U-Net architecture for rain segmentation in images. The model takes a 352x352 rain image and returns a rain mask of the image as shown in Figure 41. The model has 4 encoder blocks, and 4 decoder blocks similar to a traditional U-Net [1, 2]. The key innovation of this model is the pretrained Hiera block in the encoder block of the model. The Hiera architecture is a hierarchical vision transformer that outputs four different channel resolution (144, 288, 576, 1152) and is from the pretrained state-of-the-art Segment Anything Model 2 [3]. All the components of the model are trained except the Hiera block. The weights of the Hiera block are frozen because we utilize the pretrained weights which is more memory-efficient than training from scratch. An adapter layer is added before each Hiera block to ensure efficient fine-tuning and encoding the features from the rain data since the Hiera block has not been trained on any rain data [2]. The adapter contains a downscale, activation function (GeLU), upscale, and GeLU to sufficiently learn to encode the features of the input image. After each encoder block the height and width resolution of the image are halved (88, 44, 22, 11) while the channels are doubled (144, 288, 576, 1152). Aggregating these multi-channel tensors is difficult which is why receptive field blocks (RFBs) are used to enhance the features by having a fixed channel size of 64. The neck of the model outputs a 64x11x11 tensor which is upsampled and concatenated with the previous RFB output of dimension 64x22x22. This is repeated until we have rain mask of the same resolution as the input image.

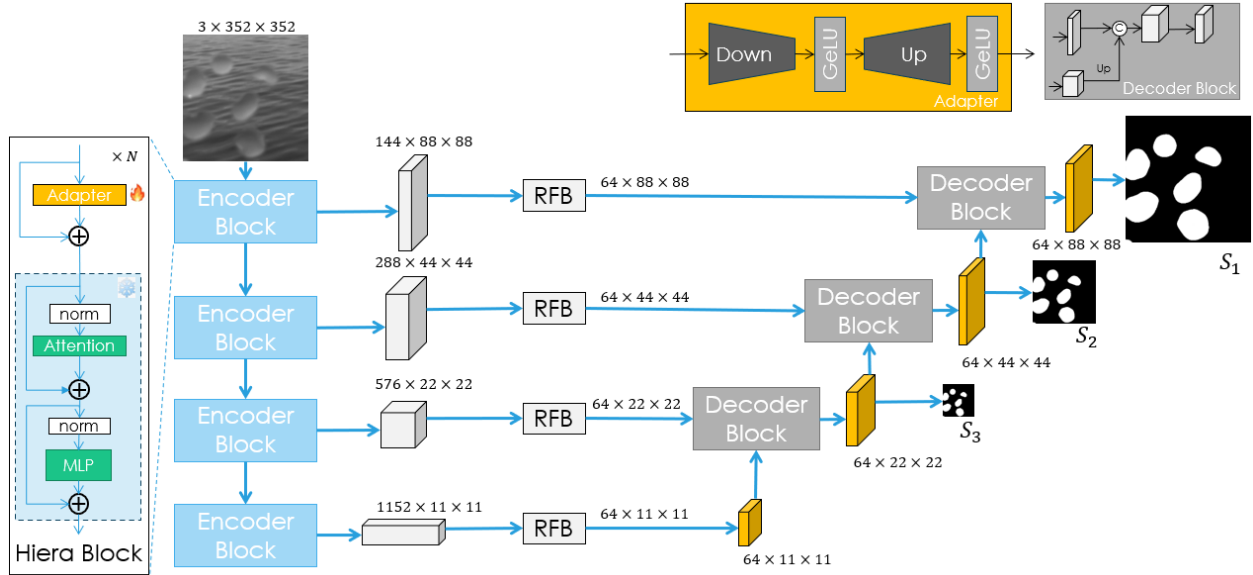


Figure 41 Segmentation Model Architecture

The model uses a weighted IoU loss and binary cross-entropy (BCE) loss across the three different rain mask outputs of the model. The model hyperparameters include a batch size of 16, learning rate of 0.0002, AdamW optimizer, and cosine annealing learning rate scheduler. The AdamW optimizer is used for finetuning transformers and the cosine annealing learning rate scheduler adjusts the learning rate based on a cosine curve. The scheduler starts with a high learning rate, the learning rate is gradually reduced following a cosine function, and then it restarts back at a high learning rate (warm restart) [4]. The model is trained for 500 epochs and on a 1280 sample rain dataset to train the model for rain segmentation in images. The loss curve shows that the loss is going down and model fits the training dataset well at around 500 epochs as shown in Figure 42.

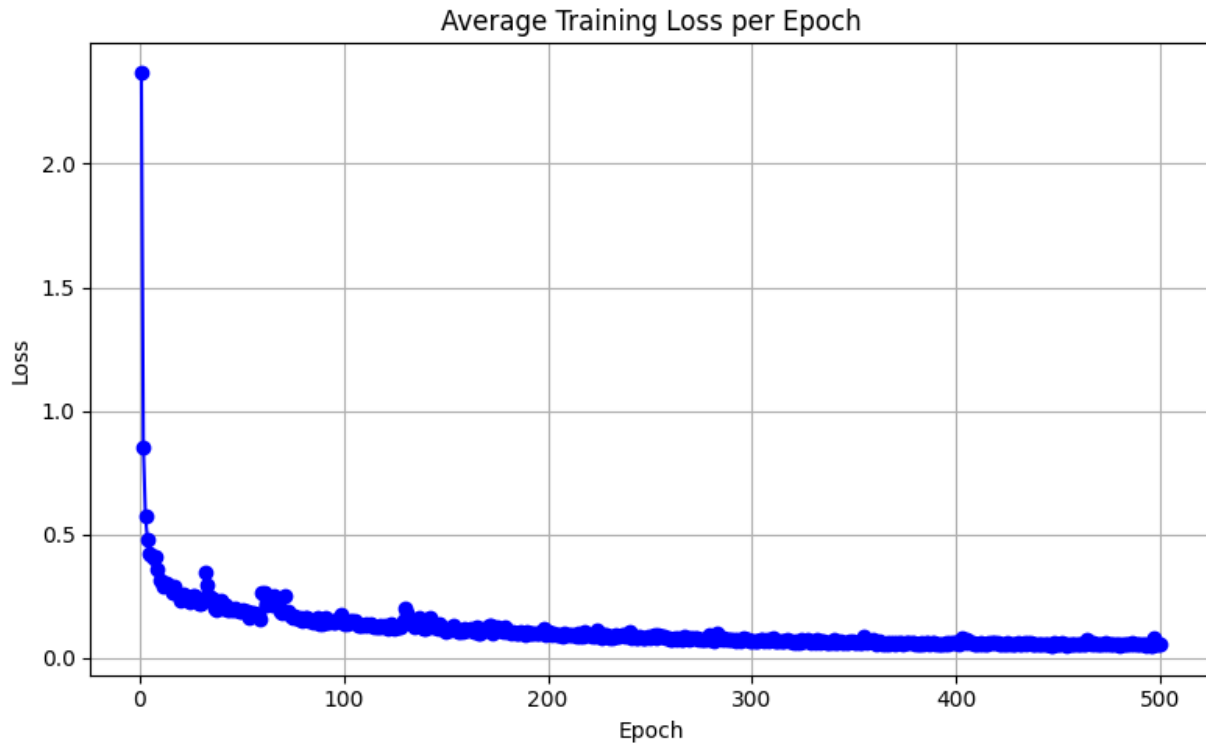
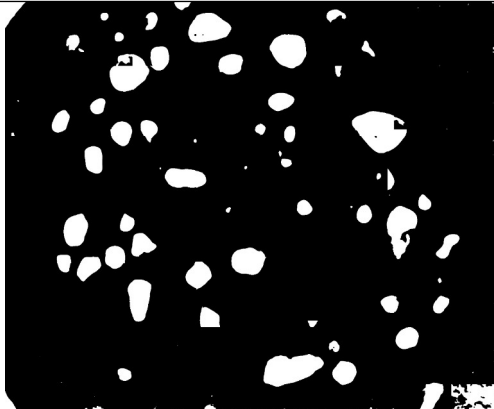
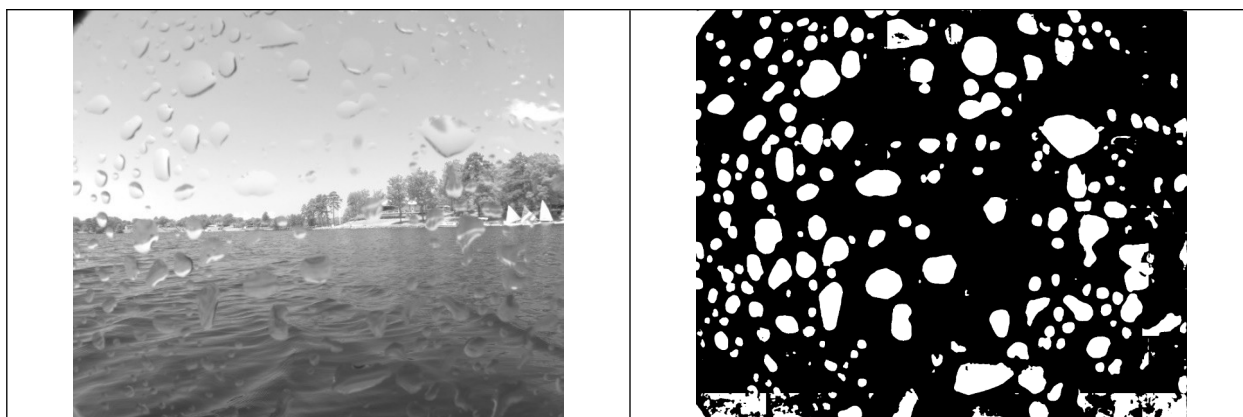


Figure 42 Segmentation Model Loss Curve

We evaluated our trained models on test data as shown in Table 2. The model performance could be improved for our test cases. Some segmentations are incomplete and tight clusters of raindrops are challenging cases for the model. We have found that this issue is related to the data labeling and data diversity issue. Cleaning the training data further and using more data from our data captures can further improve our segmentation model.

Table 2 Segmentation Results on Test Data

Input Image	Rain Mask
	

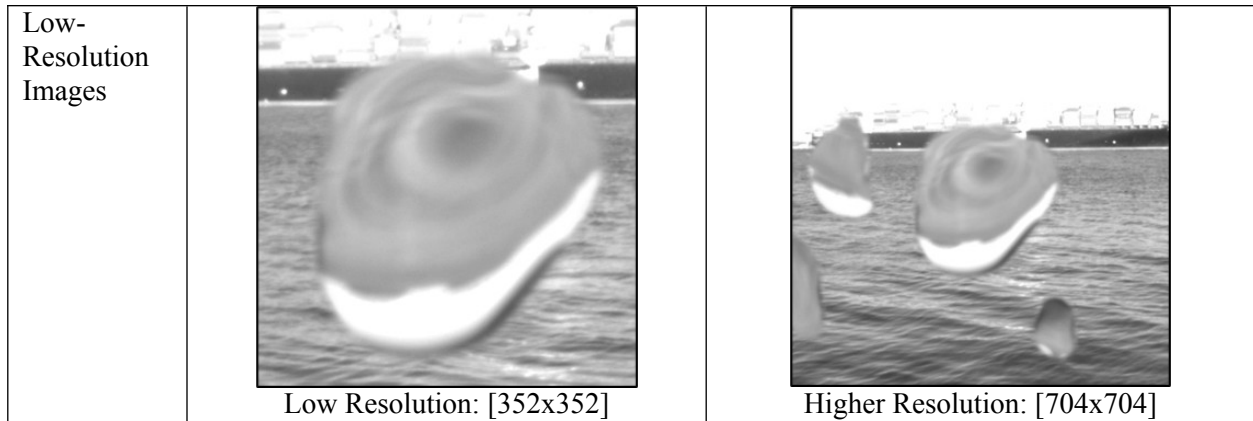


Part 2: Dataset Creation

Creating a robust dataset is equally as important to designing model architecture. Upon further inspection the training data had limitations such as false labels, incomplete segmentation labels, and low-resolution images as shown in Table 3. False labels were due to the intensity matching step introduced in the synthetic data creation pipeline. Some incomplete segmentation labels were also present in our training data which can affect model performance. We also saw that smaller resolutions for training could affect the model's ability to distinguish between background and foreground details when introduced to larger raindrops that cover most of the 352x352 image plane.

Table 3 Training Dataset Limitations

False Labels		
Incorrect Labels		



We have developed a new rain dataset cleaning procedure to ensure accurate ground truth labeling for our training dataset as shown in Figure 43. Firstly, we use a segmentation tool to do a coarse segmentation of raindrops based on a rough area of interest in the raindrop image (i.e. raindrops in various water and sky areas). Once we have our rain mask, we use a connected component analysis to find the centroid of each of the segmented raindrops. Then the next step is to iterate over each of these centroids and crop a 704x704 window. These 704x704 crops are then segmented once more with finer detail so that we have a 1-to-1 correspondence between the raindrop image crop and the rain mask.

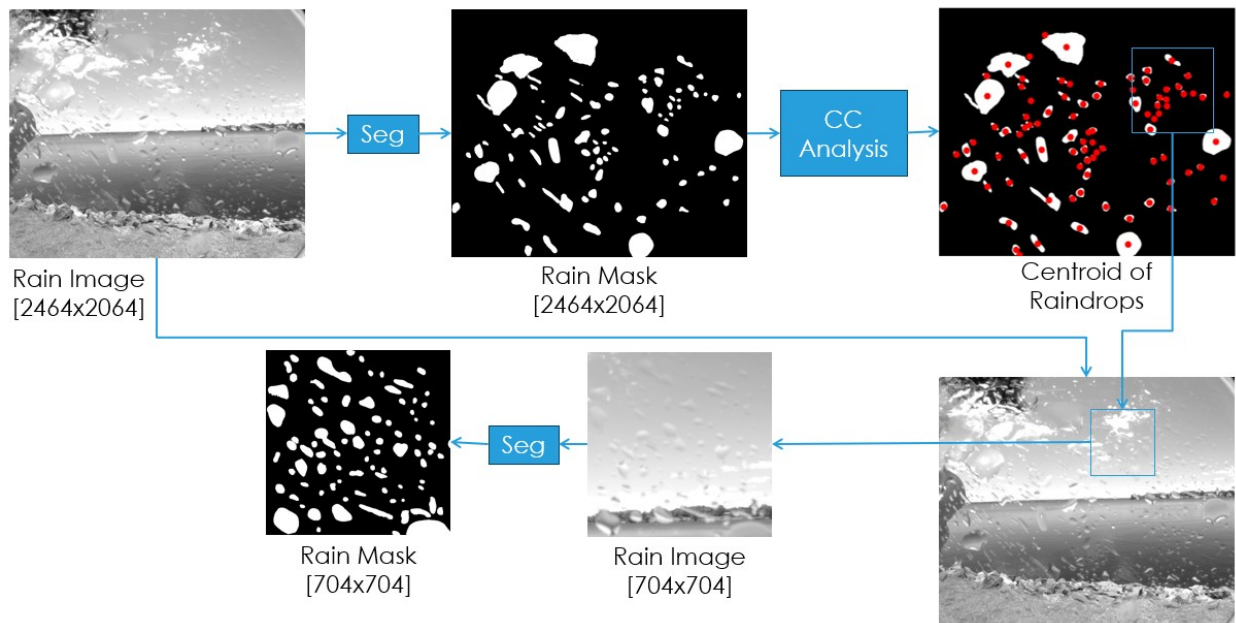
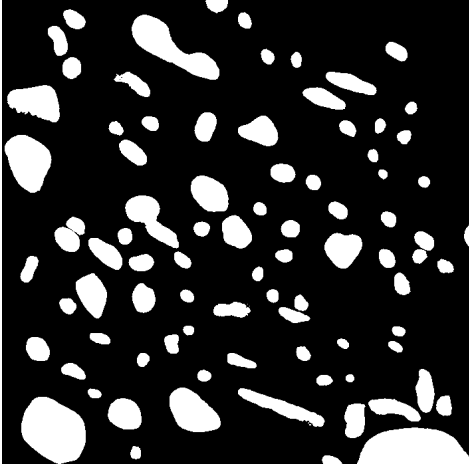


Figure 43 Data Cleaning Pipeline

We addressed the false label issue by using raw rain data instead of synthetic data since rain segmentation does not require clean images for training. We addressed the incomplete segmentation label issue by implementing an iterative segmentation procedure that segments all raindrops in the image plane. Lastly, we increase the image size of the dataset to 704x704 so that the segmentation model can benefit from more contextual and raindrop boundary information. We currently have ~600 samples of accurately labeled data. An example of a rain image and rain mask pair can be seen in Table 4. We see that the raindrops in the rain image and the white pixels in the rain mask have a 1-to-1 correspondence.

Table 4 Training Data Example

Rain Image (Input)	Rain Mask (Ground Truth)
	

2.2.6 Geometric Range Estimation

Last month, we investigated the effect that camera calibration has on geometric range estimation. Furthermore, we also investigated the effect the IMU (and subsequently when we pull the IMU value) has on range estimation. This month we continued the investigation into calibration and made significant progress in improving the calibration of our perception systems. We successfully tested and evaluated multiple calibration methods, gaining valuable insights into how each approach impacts system performance. Notably, we implemented a joint calibration procedure for the camera and IMU, obtaining both a measured transformation matrix and a time offset between the two sensors.

Over the course of the month, we conducted four distinct calibration methods: **close**, **mid**, **far**, and **combined**.

- **Close calibration (0-5m)** involved placing the calibration board a few meters from the camera and moving the camera (left, right, and tilted) to gather sufficient IMU variation.
- **Mid calibration (5-10m)** followed a similar procedure, with the board positioned approximately five meters away.
- **Far calibration (10-15m)** extended the distance to around ten meters, again with camera motion to introduce variability.
- **Combined calibration** kept the camera stationary while the target was moved between close, mid, and far positions, including tilts.

Images collected from these sessions were processed using **Kalibr**, and their intrinsic parameters and distortion coefficients were compared. Among the four methods, **far calibration** produced the lowest reprojection error. We suspect this is due to the calibration board's pattern being smaller in the image at greater distances, providing a more accurate representation of the scene. This result suggests that board scale and target distance may play a significant role in calibration quality, which we plan to explore further in our ongoing work to refine the calibration procedure for minimal reprojection error. Previous work on this contract for Task 1.4 on uncertainty suggests we may need to consider longer ranges (possibly as long as 20 meters) to ensure we are conducting the calibration at ranges where the factors that influence the lens modulation transfer function (MTF) have stabilized.

Additionally, we successfully incorporated the IMU into our system calibration. Using Kalibr, we first calibrated the camera alone, then performed a joint calibration with the IMU. This process yielded a

transformation matrix that accounts for the positional and rotational offsets between the camera and IMU, improving the accuracy of sensor fusion tasks such as ray tracing. These small offsets become increasingly important when dealing with distant objects, where even a single-pixel discrepancy can result in large depth estimation errors. We also obtained a **time shift** between the camera and IMU, giving us a better understanding of when camera frames are captured relative to IMU measurements. This synchronization allows us to align IMU data more accurately with visual frames, ultimately enhancing depth estimation. Next month, we aim to refine our calibration procedures and fully integrate these improvements into the perception system to support more precise and reliable depth sensing. It is crucial that we calibrate our cameras in a way that is optimized for our system, and we hope to finalize that method in the coming months.

2.2.7 Longwave Infrared (LWIR) Range Estimation

Last month, the team trained the Infrared Ranging Algorithm, XGBoost, with 269 LWIR images paired with ground truth distance datapoints. The training variables included the x-pixel of the object, the y-pixel of the object, and the difference in intensity of water particles near the object and water particles near the sensor. Due to initial parameterization and lack of sensor gain as a training variable, the algorithm did not demonstrate any robustness to variability in environment. The infrared ranging algorithm performed decently when trained and tested in comparable environments; however, the algorithm performed poorly when it was trained and tested with datapoints that combined mid-day, sunny, cloudy, and night-time scenes.

This month, Integer explored methods to increase the robustness and reliability of the infrared ranging algorithm. The primary theory for the lack of robustness in the algorithm was because the LWIR sensors that collected the open-source data utilized Automatic Gain Control (AGC). AGC directly impacted the fluctuation of intensity values within each image from scene to scene based on ambient environment temperature ranges, regardless of the actual temperature of the water particles. In the middle of the day, the AGC caused water particles to have a much smaller change in pixel intensity as a function of distance. Conversely, in the evening, the AGC caused water particles to have a larger pixel intensity difference as a function of distance. This was due to a larger range of received infrared radiation during the middle of the day, where the sun is heating objects in the background to a much higher temperature. Not knowing the value of the sensor gain in each image made it impossible for XGBoost to accurately predict the range to an object based solely on the pixel intensity difference and x/y-pixel location of object in-frame. Utilizing AGC in an infrared ranging algorithm is only a viable method if the gain value is quantified and parameterized into the algorithm. Shown below in Figure 44 is a visualization of the impact that AGC has on pixel intensity gradient of the sea surface from two vastly different scenes.

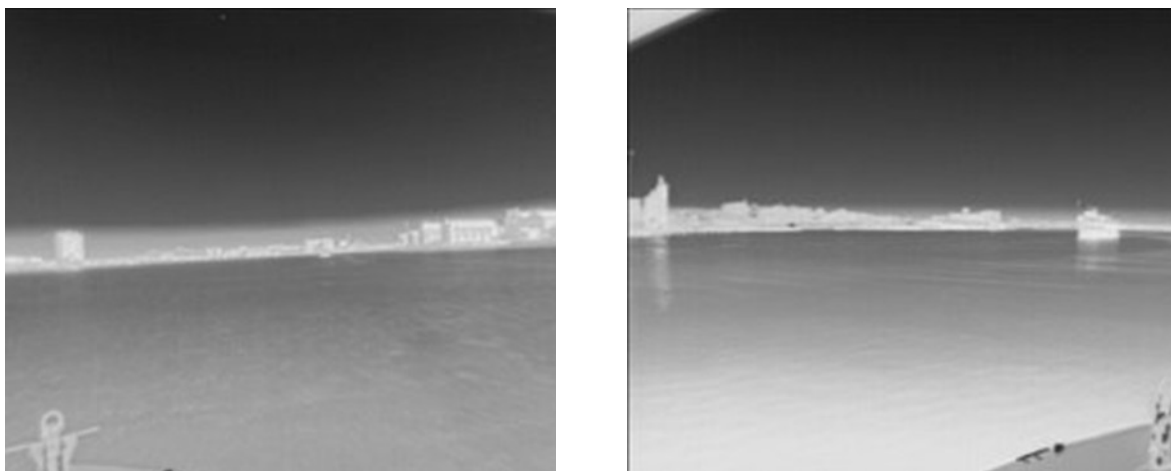


Figure 44. Two LWIR images from two different datasets. The image on the left was taken at mid-day, while the image on the right was taken at night. The sea surface in both images is isothermal, but the range of temperature in the ambient environment is vastly different. These images illustrate the effect of Automatic Gain Control on the pixel intensity gradient of the sea surface.

This month, Integer also explored the optimization of training variables for the infrared ranging algorithm. In order to parameterize a remedy to solve the issue of the open-source datasets failing to contain sensor gain metadata, the pixel intensity difference between the water particles near the object and the water particles near the sensor were parsed into two additional independent training variables. Instead of only 3 independent training variables (x-pixel, y-pixel, and pixel intensity difference), there are currently 5 independent training variables, shown in Table 5.

Table 5. Infrared Ranging Algorithm's independent variables and their respective weights

<i>Variable</i>	<i>Description</i>	<i>Weight (Feature Importance)</i>
<i>X_pixel</i>	Annotated x-pixel coordinate of object	3%
<i>Y_pixel</i>	Annotated y-pixel coordinate of object	26%
<i>Intensity_Object</i>	Pixel intensity of water particles around object	46%
<i>Intensity_Sensor</i>	Pixel intensity of water particles near LWIR optic	12%
<i>Intensity_Delta</i>	Difference between object & sensor pixel intensities	13%

Separating the two intensity values into their own independent variables enabled the algorithm to rudimentarily parse the gain of the sensor in each particular image; this reparameterization increased the R^2 value from 0.60 to 0.90. These results, shown in Figure 45, quantify the necessity of incorporating sensor gain as a training variable in the thermal ranging algorithm.

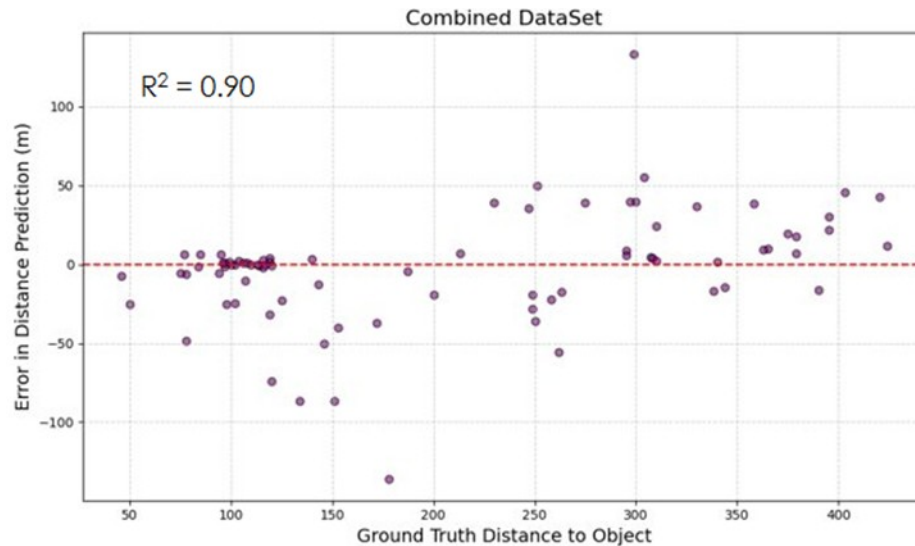


Figure 45. Infrared Detection Algorithm's residual error in range estimation (m) plotted against the ground truth distance (extracted from radar paired data) to the algorithm's respective range predictions

In preparation for self-procured LWIR maritime data collection, Integer demoed an LWIR camera from Axiom Optics, the Niels12 SXGA 1280x1024 model. The team explored the capability of the optic and its applicability in the infrared ranging method as well as object detection. During the demo, Integer also collected more than 250 maritime LWIR images paired with ground truth distances of waterborne objects at a local lake. During this data collection, the following metadata was collected:

- Object Description
- Epoch and Timestamp (including time/date)
- Sensor Gain
- Sensor Angle
- Sensor Temperature
- Water Temperature
- Air temperature
- Humidity
- Wind Speed & Direction
- General conditions (sunny, cloudy, rainy)

This initial data collection with a heavy emphasis on metadata poises the team for algorithm re-parameterization as more data is collected in various atmospherics and environments.

Next month, the team plans to explore image stacking and temporal filtering in LWIR images in order to reduce object-emitted surface-reflected radiated noise. Additionally, the team plans to select and purchase an LWIR optic to continue building upon our LWIR maritime dataset with precisely recorded metadata.

2.2.8 Deep Learning-based Range Estimation

The objective of the deep learning methods for range estimation is to assess its viability as a potential alternative for outdoor, open scene range estimation. In previous exploration periods (September 2024)

Integer has worked with transformer architectures for computer vision tasks and has tested their potential in zero shot applications with no custom training. The implementation of these models yields excellent results, however, this performance comes at a significant computational cost. Building on the experience acquired, Integer decided to test a transformer model as a possible solution for the monocular depth estimation (MDE) problem in open-world scenes.

Monocular depth estimation is a computer vision task that aims to predict the depth of objects in a scene. When working with models which aim to solve this problem there are generally two types of solutions: absolute depth and relative depth. Relative depth is the most common approach. Relative refers to the ratio between two points, the model determines the result based on comparison between different reference points. The absolute, also known as metric depth estimation, determines the depth of the object on a physical scale, normally the metric system. The model used for this investigation was Depth-Anything-v2 [5]. Integer was particularly interested in this model primarily because of their initial fully synthetic high accuracy training schema and their label-level distillation technique using pseudo labels. The model encoder is first trained on purely synthetic data with high-accuracy and high-granularity labels. Next, the trained encoder is used to create labels of real images, and the student model is then trained on these pseudo-labels. An example of the training pipeline is shown in Figure 46 and some examples of the model performing in an indoor environment are shown in Figure 47.

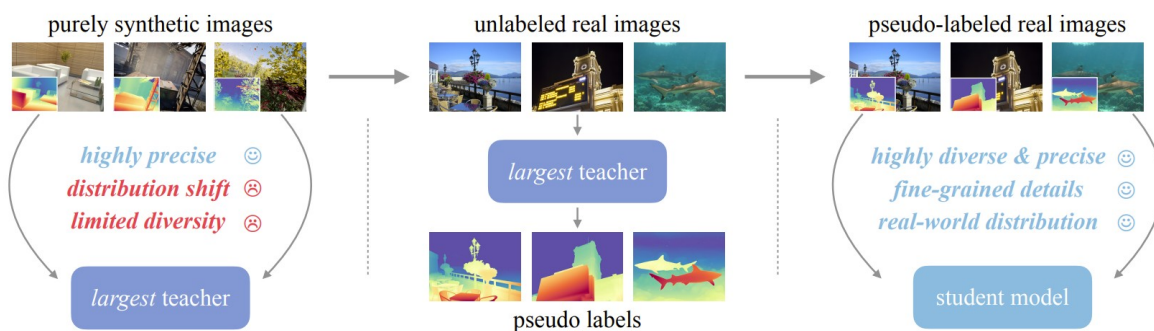


Figure 46. Training process and pseud-label creation of Depth-Anything-V2 [reference]

In addition, the encoder is fine-tuned for outdoor depth estimation using Virtual KITTI [6], a digital version of a dataset originally created using cloud-points and image pairs.



Figure 47. Example of Anything-Depth-V2 output. Left, original images. Middle, relative depth output. Right, metric depth output the color value represents the estimated pixel depth in meters.

The weight set used for outdoor estimation was *depth_anything_v2_metric_vkitti_vitl*. The model input size used was 1036 px and the maximum distance was 80m. The regular geometric methods used for range estimation rely heavily on camera parameters and orientation: focal distance, inclination, tilt. The model does not take any input parameter for these variables; therefore a few tests were conducted to measure the impact of roll, pitch, and yaw. Table 6, Table 7, and Table 8 show the model prediction under different conditions.

Table 6. Metric depth model performance with target at 25m

Pitch (°)	Prediction (25m)	Standard Deviation
-2°	28.36 m	0.13 m
0°	28.13 m	0.08 m
2°	27.72 m	0.08 m

Table 7. Metric depth model performance with target at 75m

Pitch (°)	Roll (°)	Yaw (°)	Prediction (75m)	Standard Deviation
-2°	0°	0°	71.76 m	0.93 m
0°	0°	0°	69.79 m	0.62 m
0°	0°	30°	67.56 m	0.29 m
2°	0°	0°	68.83 m	0.39 m

Table 8. Metric depth model performance with target at 50m

Pitch (°)	Roll (°)	Yaw (°)	Prediction (50m)	Standard Deviation
0°	0°	0°	47.55 m	0.82 m
0°	0°	4°	51 m	0.39 m
0°	4°	4°	49.47 m	0.42 m
2°	0°	0°	46.32 m	0.59 m
2°	4°	0°	49.82	0.45 m
2°	4°	4°	46.09	0.58 m

The model is capable of estimating an object's distance with an error smaller than 0.2m (0.4%). The model is sensitive to pitch, roll, and yaw, which shows it can increase the error by up to 4m. Error increases as the distance approaches the maximum distance parameter. The model performs more accurately at mid ranges relative to the maximum distance (50m).

The results from the deep learning method were exceptional considering no custom training was performed. A custom training job could enhance accuracy and extend the estimation range if the dataset is comprehensive enough and meticulously curated. The creation of a synthetic set for custom training is subject to the exploration of new tools and techniques for it to be consolidated. The time and effort required to develop a custom dataset tailored to the model's specific needs are acknowledged as significant but will be addressed in future work.

2.2.9 Ray Tracing Range Estimation Uncertainty

During this reporting period, the Integer Team studied the uncertainty associated with estimating range using the Ray Tracing approach. It is important to understand the fidelity of range estimates made using ray tracing so that we know the limitations of the approach, and what system modifications could be effective in increasing the fidelity of the measurements. The predicted uncertainty of the range estimates is derived from the errors and uncertainties inherent in the systems we are using; namely, the camera, lens, and IMU, as well as the environment. For the moment, we limit the analysis to two-dimensions to keep things relatively simple and ignore the environment (assume no optical turbulence and a flat calm ocean).

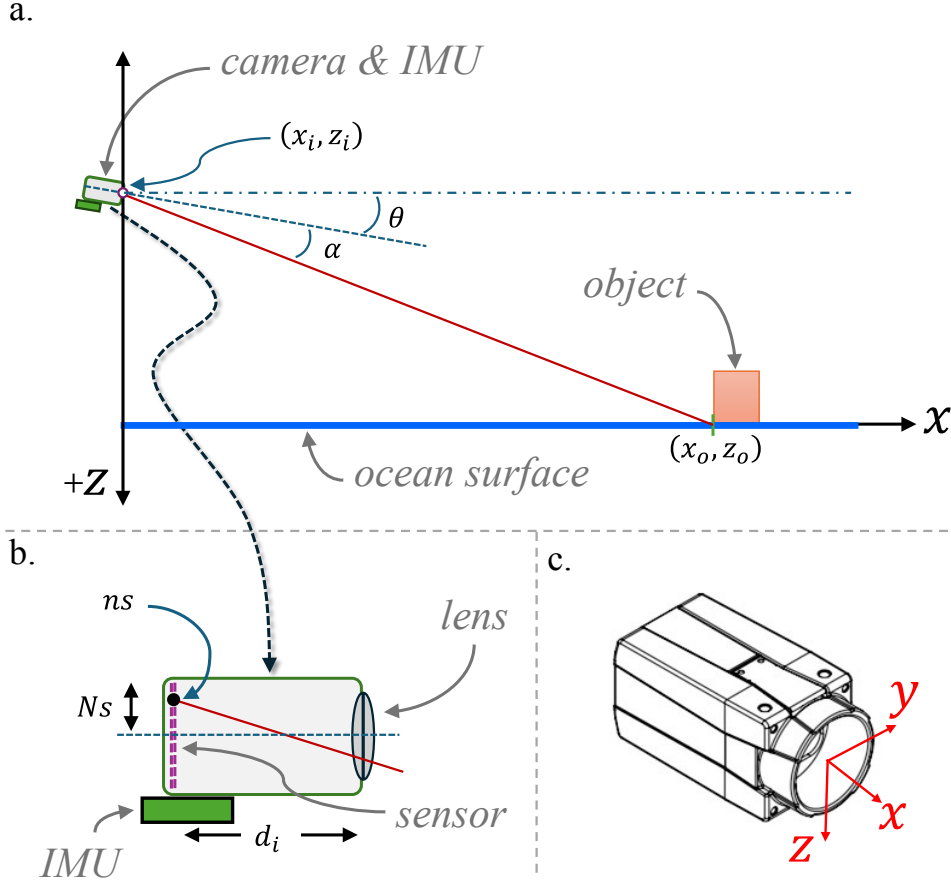


Figure 48. The variables and coordinates used in the two-dimensional uncertainty analysis. (a) is the coordinates of the external system where θ is the pitch angle of the camera and α is the incidence angle of the ray that emanates from where the object intersects the free surface of the ocean. (b) shows a cross-section of the camera where s is the height of the pixels and $2N$ is the number of pixel rows in the camera sensor. d_i is the distance from the lens to the camera. For thin lens assumptions and large working distances, $d_i \approx f$, where f is the focal length. (c) shows the 3-axis coordinate system (it is consistent with (a)).

Using the variables shown in Figure 48, we define the range starting with the equations for α and θ ,

$$-\tan(\alpha + \theta) = \frac{z_i - z_o}{x_i - x_o} \quad (1)$$

$$\tan \alpha = \frac{ns}{f}, \quad (2)$$

and then substitute (2) into (1) and rearrange to solve for the range:

$$r = x_o - x_i = (z_i - z_o) \left[\frac{f - ns \tan \theta}{ns + f \tan \theta} \right]. \quad (3)$$

Standard engineering practice is to define system uncertainty as the square root of the square of the independent variable uncertainties,

$$U_r = \sqrt{\left(\frac{\partial r}{\partial z_i} \right)^2 \varepsilon_{z_i}^2 + \left(\frac{\partial r}{\partial z_o} \right)^2 \varepsilon_{z_o}^2 + \left(\frac{\partial r}{\partial f} \right)^2 \varepsilon_f^2 + \left(\frac{\partial r}{\partial n} \right)^2 \varepsilon_n^2 + \left(\frac{\partial r}{\partial s} \right)^2 \varepsilon_s^2 + \left(\frac{\partial r}{\partial \theta} \right)^2 \varepsilon_\theta^2}, \quad (4)$$

where ε is the error or uncertainty for each. We first solve for the six partial differential equations:

$$\frac{\partial r}{\partial z_i} = \left[\frac{f - ns \tan \theta}{ns + f \tan \theta} \right], \quad (5)$$

$$\frac{\partial r}{\partial z_o} = - \left[\frac{f - ns \tan \theta}{ns + f \tan \theta} \right], \quad (6)$$

$$\frac{\partial r}{\partial f} = \frac{ns(z_i - z_o)}{\cos^2 \theta (ns + f \tan \theta)^2}, \quad (7)$$

$$\frac{\partial r}{\partial n} = \frac{-sn(z_i - z_o)}{\cos^2 \theta (ns + f \tan \theta)^2}, \quad (8)$$

$$\frac{\partial r}{\partial s} = \frac{-nf(z_i - z_o)}{\cos^2 \theta (ns + f \tan \theta)^2}, \quad (9)$$

$$\text{and } \frac{\partial r}{\partial \theta} = \frac{(z_i - z_o)(f^2 + (ns)^2)}{\cos^2 \theta (ns + f \tan \theta)^2}. \quad (10)$$

Again, convention is to use a 95% confidence interval (CI) for the error terms. Some of the terms are provided for the IMU, camera, and lens by the manufacturer, but they are not always clear if the values represent 95%CI. In cases where they are providing maximum limits, we assume that to be equivalent to 6σ and then estimate the 95%CI as $1.96(x_{limit}/6)$ and the mean as the middle of the limiting values.

Although the field of view provided by the lens is continuous, the camera sensor discretizes the input by pixels. Therefore, we must combine the camera errors and lens errors to find the pixel, n_d , that senses the ray that represents the range to the object as shown in Figure 49.

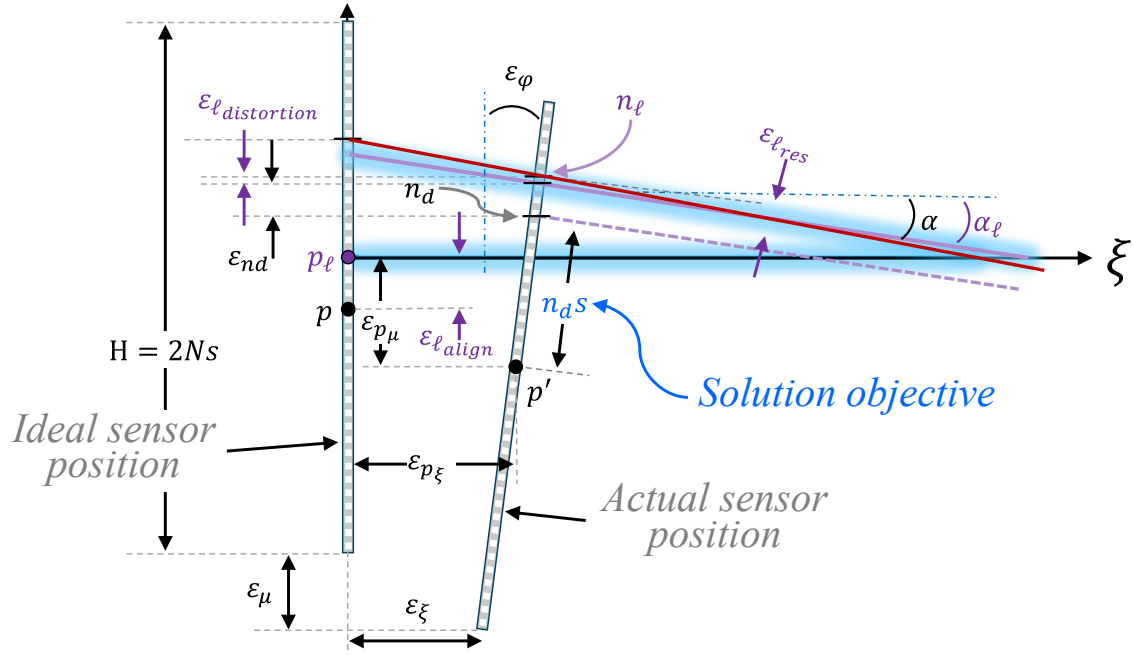


Figure 49. The camera and lens uncertainties that influence the location of n_d , the pixel row sensing the range. The figure shows the errors due to the camera sensor not being perfectly aligned with the back wall of the camera (ϵ_ξ , ϵ_μ , ϵ_ϕ), and imperfections in the lens ($\epsilon_{\ell_{\text{distortion}}}$, $\epsilon_{\ell_{\text{align}}}$, $\epsilon_{\ell_{\text{res}}}$). The fuzzy blue is intended to represent lens resolution and the dashed purple line the effective incidence angle; that is, the angle the camera sensor attributes to the location of object-free surface intersection.

We see that the lens imperfections not only influence the pixel that senses the range, but also the principal point which is the reference for determining α ($\alpha = \tan^{-1}(n_d s / f)$).

Monte Carlo simulations can be used to quantify the uncertainty, however, initial studies suggest that the pitch angle uncertainty will be one of the most significant contributors unless the IMU uses RTK GPS. Additionally, in studying camera Modulation Transfer Function (MTF) functions, we have confirmed that it is range dependent, with the change in behavior not reaching single digit percent until a range of five to 20 meters for short focal length lenses (strongly dependent on lens design and quality). This indicates calibration of the camera is range dependent and may need to be studied further if we wish to accurately quantify image uncertainty. If useful, future investigations will add environmental factors and/or three-dimensional effects to the analysis.

2.2.10 Shoreline Characterization

This month, Integer began an investigation on potential shoreline surveillance approaches with the goal of autonomously making go/no-go decisions about beach landings. In shoreline characterization for beach landings, image segmentation is a promising approach that can be used to accurately identify known objects and potential hazards that appear on the shore zone. The pixel level detail from segmentation [7] can accurately map densely packed hazards, such as rocks [8] or vegetation [9]. Fine grained knowledge of shoreline composition via segmentation is valuable for assessing safe beach landing zones.

The ideal segmentation model balances accuracy, efficiency, and hardware constraints. Models like SAM2 [10], SegFormer [11], and YOLOv7 [12] are state of the art segmentation models that offer different strengths. At a high level, SAM2 (Segment Anything v2) provides high quality, prompt-based segmentation. An example of a SAM2 inference is shown in Figure 50. Despite achieving strong shoreline segmentation in this example, the base model was found to be computationally heavy, making it less practical for real time inferencing on lightweight edge devices. On the other hand, YOLOv7 for segmentation offers fast inference with strong performance and continues to be a reliable semantic segmentation model to use, but the advent of transformers has made the YOLOv7 architecture slightly dated. In contrast, SegFormer, is a lightweight transformer-based model designed by NVIDIA for semantic segmentation [13]. It notably has a strong balance between speed, accuracy, and resource efficiency, making it particularly well suited for shoreline segmentation tasks, specifically on NVIDIA hardware. A performance plot in Figure 51 compares the mean Intersection of Union [14] (mIOU) metrics obtained on the ADE20k dataset [15] from various segmentation models, such as SegFormer, DeepLab [16], FCN [17], and Swin transformers [18]. The figure highlights that SegFormer is shown to achieve the highest performance on the ADE20k dataset (51.0% mIOU) while also having significantly less parameters than other models. This, in tandem with intrinsic compatibility with NVIDIA devices, makes SegFormer a good starting model to work with.

To train a SegFormer model for shoreline hazard segmentation, a segmentation dataset containing shoreline data is needed. While ocean related imagery is widely available online, data specifically depicting a boat approaching a shoreline, from the perspective of the boat, is scarce. This gap between what is needed and what is openly available highlights the value of designing a custom, ocean world environment using a video game engine. A virtual ocean environment was created on Blender [19], with a camera attached to the simulated boat. An example image captured from this scene is shown in Figure 52. In this example, the boat was animated to follow controlled paths, effectively simulating real world scenarios (e.g., a boat moving towards the shore). By extracting individual frames and generating binary masks for each object layer, a large, customizable segmentation dataset can be created. Examples of these binary masks are shown in Figure 53. These masks form highly accurate, pixel level annotations that simulate complex beach landing scenarios, such as the presence of hazards on the beach. This synthetic dataset fills a crucial gap in open-source shoreline data and ultimately enables precise training and benchmarking of segmentation models in realistic, but controlled conditions.

In conclusion, a Blender-based shoreline simulation has enabled the creation of a tailored, boat-approaching-shoreline segmentation dataset that addresses a critical gap in open-sourced training data for real time shoreline characterization. This simulation-to-annotation pipeline has laid the groundwork for training efficient segmentation models like SegFormer. Future work will focus on generating a high-fidelity, Blender sourced segmentation dataset that simulates a shoreline approach scenario from the perspective of a boat mounted camera. This dataset will include diverse terrain types such as sand, rocks, vegetation, or debris, with segmentation style annotations for each frame. This will be used to later evaluate the performance of the SegFormer model in detecting and segmenting shoreline hazards.



Figure 50. Prompt based segmentation output from SAM2. Points inside specific objects of interest are manually chosen and can be used to control what object SAM2 should segment.

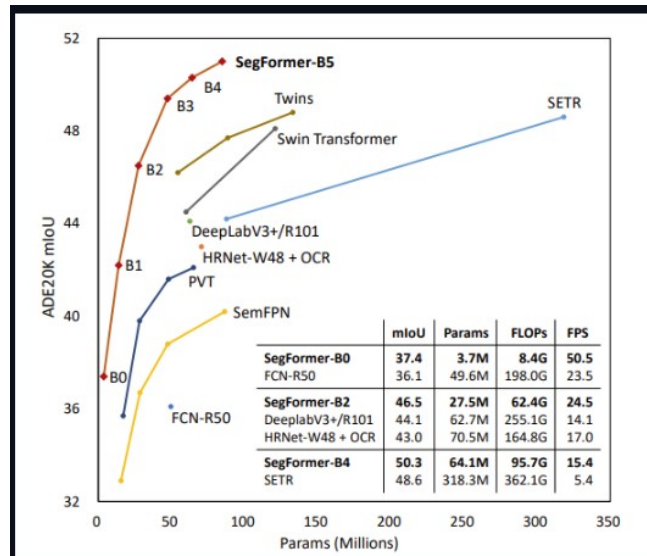


Figure 51. Performance metrics comparing SegFormer with various other segmentation models (DeepLab, FCN, etc). SegFormer is shown to achieve the highest performance on the ADE20k dataset (51.0% mIoU) while also having significantly less parameters than other methods. [11]



Figure 52. Example synthetic image generated from Blender. The image was captured by attaching a camera to the top of a vessel. The ship was animated to move towards the shoreline, and the camera was able to capture each frame as it moved towards the shore.

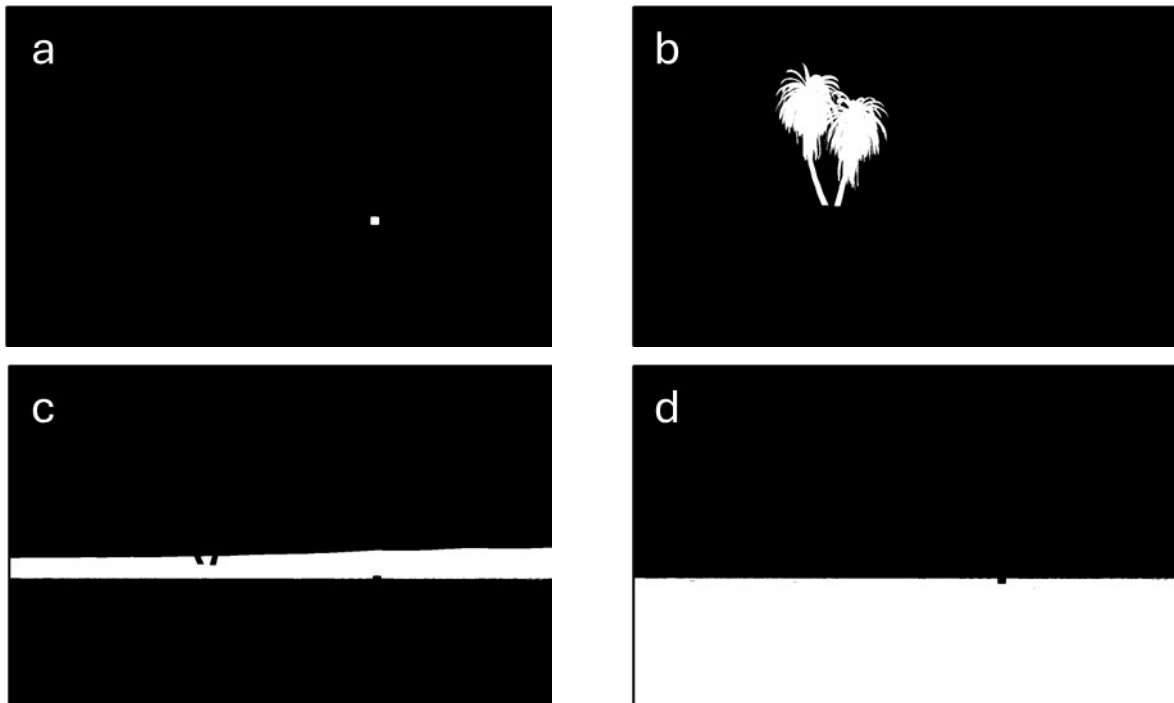


Figure 53. Subfigures (a-d) show representative binary masks generated from the example synthetic shoreline image shown previously in Figure 52. The masks represent a) Buoy, b) Obstacles, c) Shoreline, and d) Water.

3. PROBLEMS

3.1 Encountered Problems

No technical issues, concerns or delays were encountered during this reporting period.

3.2 Anticipated Problems

No current problems are expected to impede project performance or impact schedule or cost.

3.3 Resource Utilization

Table 1: Financial summary for USC contract number N00014-23-C-1052.

Budget	\$7,140,000
Cost incurred during May 2025	\$221,496
Cumulative cost incurred as of 31 May 2025	\$2,971,216
Cost to complete	\$4,168,784

4. NEXT REPORTING PERIOD ACTIONS

The efforts in the next period will be focused on:

1. Algorithms and Model development
 - Refine the FLIR camera calibration for improved IR-RGB alignment, achieve the 10,000-image annotation milestone, and continue targeted data capture sessions to enhance dataset diversity and robustness.
 - Regarding the camera-radar combined system: beyond these early milestones, we'll broaden our approach to include self-supervised learning strategies and curate a much larger, diverse dataset to drive continued improvements. Ultimately, by year five, our goal is to deliver a production-ready, low-RCS maritime-target detection system alongside the A-MARV open dataset; together enabling a more comprehensive all-weather, 24/7 maritime situational awareness.
 - We will continue to improve the rain segmentation model by 1.) making the segmentation tool faster and more efficient for data labeling, 2.) increasing our dataset size to ~2000 images with accurate ground truth labels, and 3.) training the segmentation model on the new dataset. The current segmentation tool is inefficient and time consuming because we must discard images that we cannot segment and images with tighter raindrop spreads may require multiple iterations of the segmentation method which take ~5 minutes per image. We believe that a clean and diverse dataset will improve the rain segmentation of the model thus improving raindrop removal for our deraining pipeline.
 - Next month, Integer plans to generate a Blender-based segmentation dataset (> 1000 images) that simulates a shoreline approach scenario from the perspective of a boat mounted camera. This dataset will include diverse terrain types (e.g., sand, rocks, vegetation, or debris) and segmentation style annotations (e.g., contours) for each frame. This will be used to later evaluate the performance of the SegFormer model in detecting and segmenting shoreline hazards.
2. Experiments and Testing
 - The next step for orthorectification involves integrating the INS for more accurate position measurements of the camera. Using these measurements, more in depth camera translation analysis will be conducted to map locations on a moving platform.
 - For the fused radar, camera, INS systems running on the Orin, we will conduct a field test deployment in Charleston, SC. The new fused sensor system will be tested in an outdoor operational environment using the tripod sensor plate. This test aims to evaluate the system's performance under real-world conditions, including weather and environmental clutter. Data will be collected along the shoreline to assess radar detection of stationary or slow-moving objects while verifying frame-level synchronization across the radar, camera, and INS.

- Next month, we plan to incorporate the IMU-camera transformation matrix and time shift into our current depth estimation system to determine the effects on the overall system.
- During the next reporting period, the team plans to explore image stacking and temporal filtering in LWIR images in order to reduce object-emitted surface-reflected radiated noise. Additionally, the team plans to select and purchase an LWIR optic to continue building upon our infrared maritime dataset with precisely recorded metadata.

5. REFERENCES

- [1] O. a. F. P. a. B. T. Ronneberger, "U-net: Convolutional networks for biomedical image segmentation," in *Medical image computing and computer-assisted intervention--MICCAI 2015: 18th international conference, Munich, Germany, October 5-9, 2015, proceedings, part III 18*, Springer, 2015, pp. 234--241.
- [2] X. a. W. Z. a. T. S. a. L. W. a. T. F. a. C. Y. a. L. S. a. M. J. a. L. G. Xiong, "Sam2-unet: Segment anything 2 makes strong encoder for natural and medical image segmentation," *arXiv preprint arXiv:2408.08870*, 2024.
- [3] N. a. G. V. a. H. Y.-T. a. H. R. a. R. C. a. M. T. a. K. H. a. R. R. a. R. C. a. G. L. a. M. E. a. P. J. a. A. K. V. Ravi, "SAM 2: Segment Anything in Images and Videos," *arXiv preprint arXiv:2408.00714*, 2024.
- [4] I. a. H. F. Loshchilov, "Sgdr: Stochastic gradient descent with warm restarts," *arXiv preprint arXiv:1608.03983*, 2016.
- [5] L. Yang, B. Kang, Z. Huang, Z. Zhao, X. Xu, J. Feng and H. Zhao, "Depth Anything V2," *arXiv*, 2024.
- [6] Y. Cabon, N. Murray and M. Humenberger, "Virtual kitti 2," *arXiv*, 2020.
- [7] Y. B. F. P. A. P. N. K. a. D. T. S. Minaee, "Image Segmentation Using Deep Learning: A Survey," *IEEE*, 2022.
- [8] J. Y. X. N. K. Z. a. W. M. Lili Fan, "RockSeg: A Novel Semantic Segmentation Network Based on a Hybrid Framework Combining a Convolutional Neural Network and Transformer for Deep Space Rock Images," *Remote Sensing*, 2023.
- [9] H. E. L.-H. Javier Arellano-Verdejo, "Towards sustainable coastal management: aerial imagery and deep learning for high-resolution Sargassum mapping," *Environmental Science*, 2024.
- [10] V. G. Y.-T. H. R. H. C. R. T. M. H. K. R. R. C. R. L. G. E. M. J. P. K. V. A. N. C. C.-Y. W. R. G. P. D. C. Nikhila Ravi, "SAM 2: Segment Anything in Images and Videos," *arXiv*, 2024.
- [11] W. W. Z. Y. A. A. J. M. A. P. L. Enze Xie, "SegFormer: Simple and Efficient Design for Semantic Segmentation with Transformers," *arXiv*, 2021.
- [12] A. B. H.-Y. M. L. Chien-Yao Wang, "YOLOv7: Trainable bag-of-freebies sets new state-of-the-art for real-time object detectors," *arXiv*, 2022.
- [13] NVIDIA, "NVIDIA Docs Hub," 26 August 2024. [Online]. Available: https://docs.nvidia.com/tao/archive/5.3.0/text/semantic_segmentation/segformer.html. [Accessed 2025 28 May].
- [14] HuggingFace, "Evaluate Metric," [Online]. Available: https://huggingface.co/spaces/evaluate-metric/mean_iou. [Accessed 28 May 2025].
- [15] H. Z. X. P. S. F. A. B. a. A. T. Bolei Zhou, "Scene Parsing through ADE20K Dataset," *Computer Vision and Pattern Recognition (CVPR)*, 2017.
- [16] G. P. I. K. K. M. A. L. Y. Liang-Chieh Chen, "DeepLab: Semantic Image Segmentation with Deep Convolutional Nets, Atrous Convolution, and Fully Connected CRFs," *arXiv*, 2016.
- [17] E. S. T. D. Jonathan Long, "Fully Convolutional Networks for Semantic Segmentation," *arXiv*, 2014.
- [18] Y. L. Y. C. H. H. Y. W. Z. Z. S. L. B. G. Ze Liu, "Swin Transformer: Hierarchical Vision Transformer using Shifted Windows," *arXiv*, 2021.
- [19] B. O. Community, *Blender - a 3D modelling and rendering package*, 2025.